

Lecture 01

Introduction to **Processor Design**

Instructor: Dr. Muhammad Imran

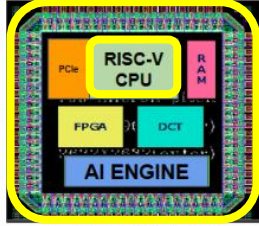


Computer Architecture | Introduction to Processor Design

Module Overview

- Abstraction Layers in Designing Computers
- RISC-V Instruction Set Architecture (ISA)
- RISC-V Microarchitecture
 - How to design a single-cycle RISC-V microprocessor?
 - How to improve the microarchitecture through pipelining?
- Dynamic Scheduling and Out-of-Order Execution
- Memory Hierarchy and Caches
- Cache Coherence in Multicores
- Virtual Memory

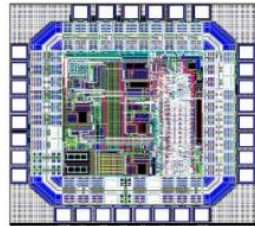
Semiconductor Supply Chain



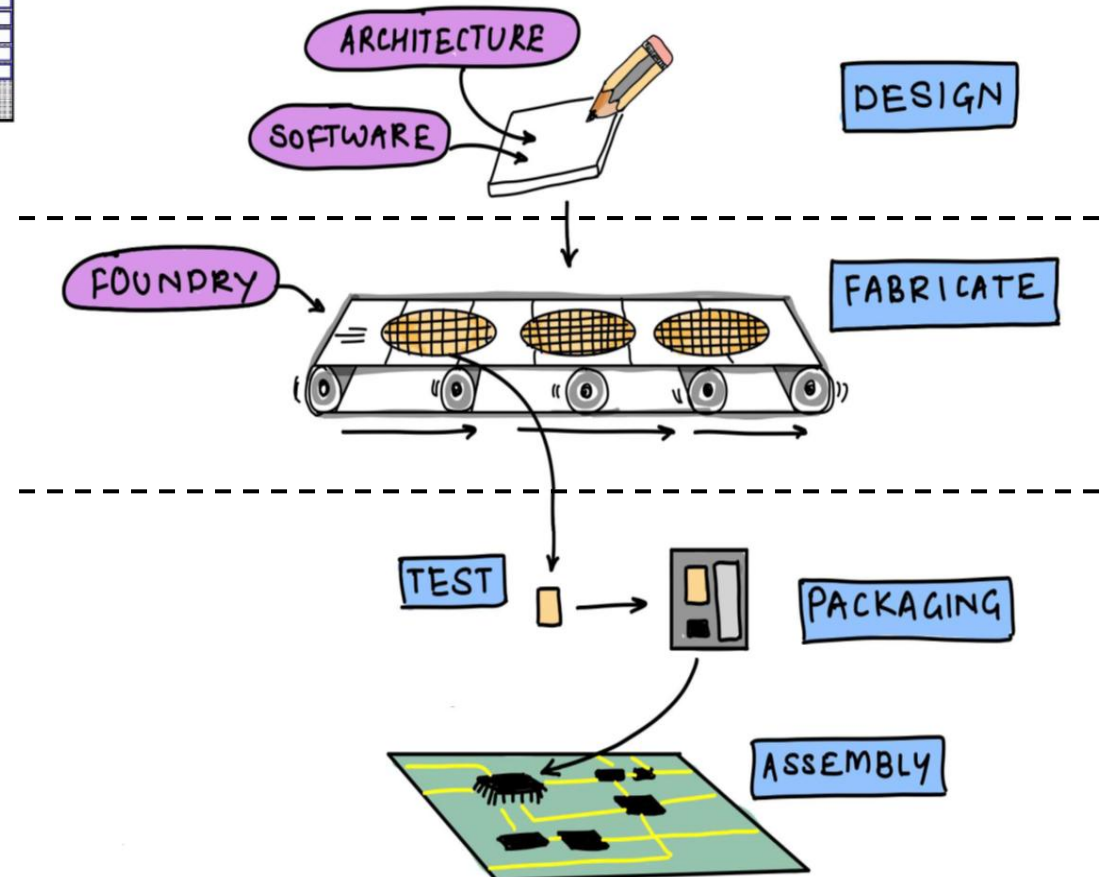
Design



Verify



Layout



Lecture 01

Introduction to **Processor Design**

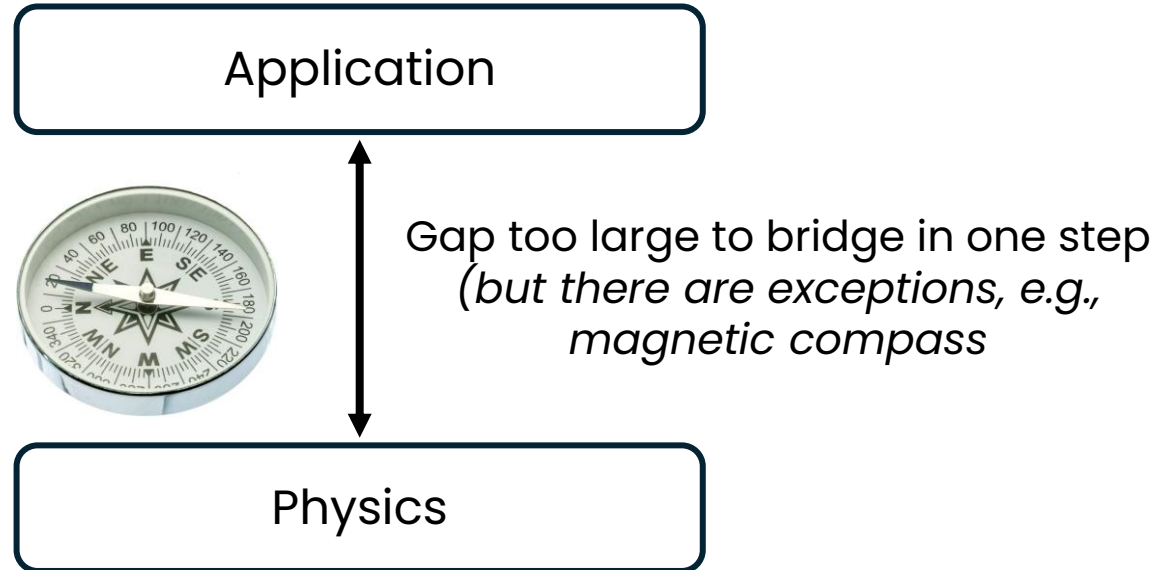
Instructor: Dr. Muhammad Imran



Contents

- Introduction to Computer Architecture and Abstraction Layers
- From Transistors to Software
- Memory
- Pipelining
- Register File

What is **Computer Architecture**?



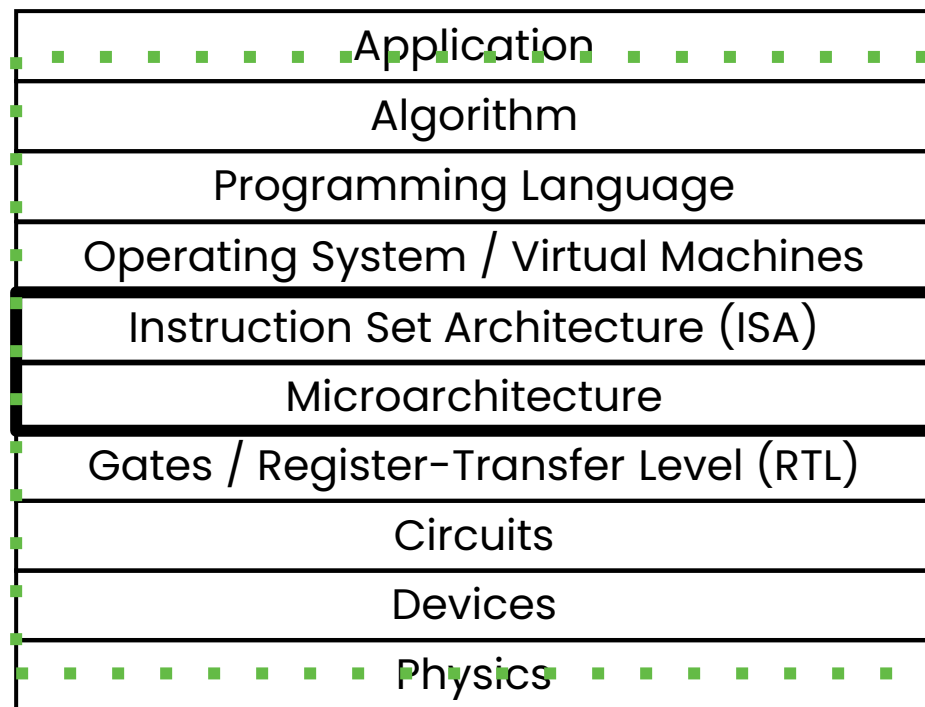
In its broadest definition, **Computer Architecture** is the design of the abstraction layers that allow us to implement information processing applications efficiently using available manufacturing technologies

Abstraction Layers

- Divide and conquer approach to designing complex digital systems!



Source: phys.org



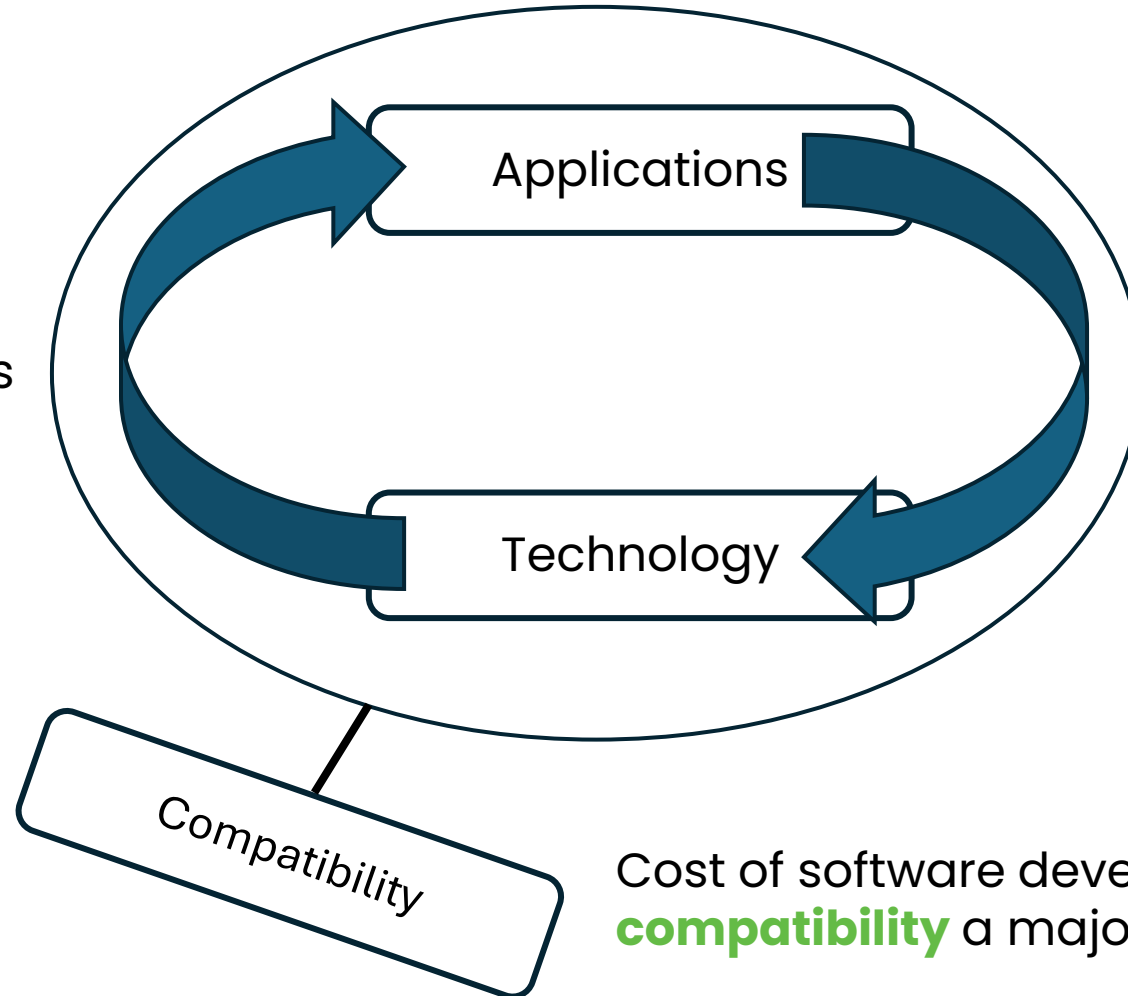
Classical view!

Modern view!

Computer Architecture is the science and art of designing computing platforms
(hardware, interface, system SW, and programming model)

Architectures are **Continuously Changing**

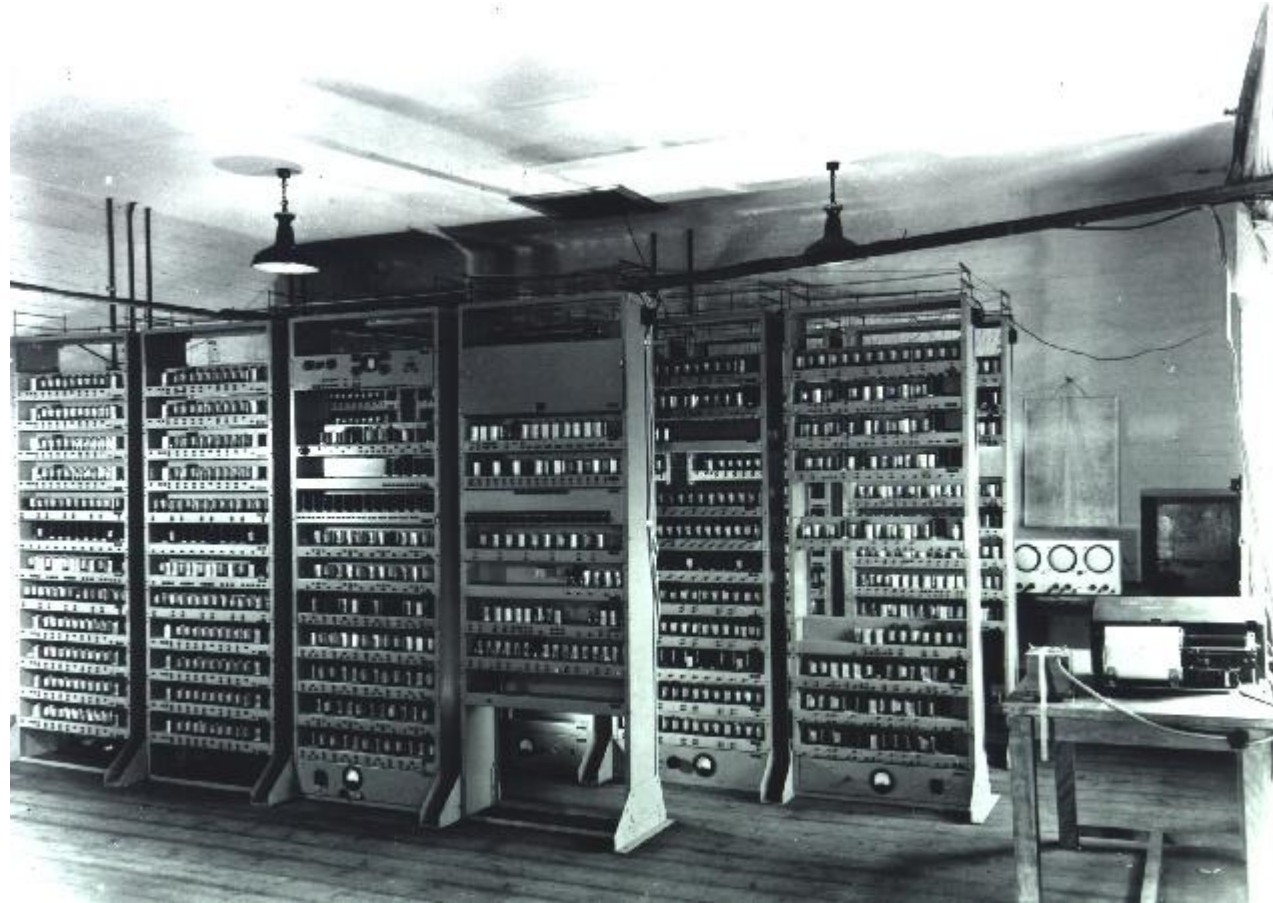
Improved Technology
makes new applications
possible



Applications suggest how
to improve technology,
provide revenue to fund
development

Cost of software development makes
compatibility a major force in the market

Computing devices **then** ...



Electronic delay storage automatic calculator (EDSAC)
(An early British Computer, 1948)

Computing devices **now** ...



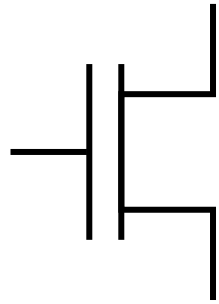
and many more ...

From Transistors **to Software**

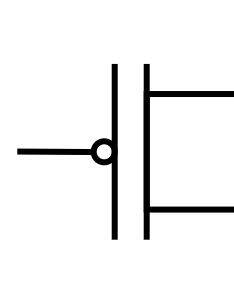


Transistor as a Switch

- MOSFET Transistors



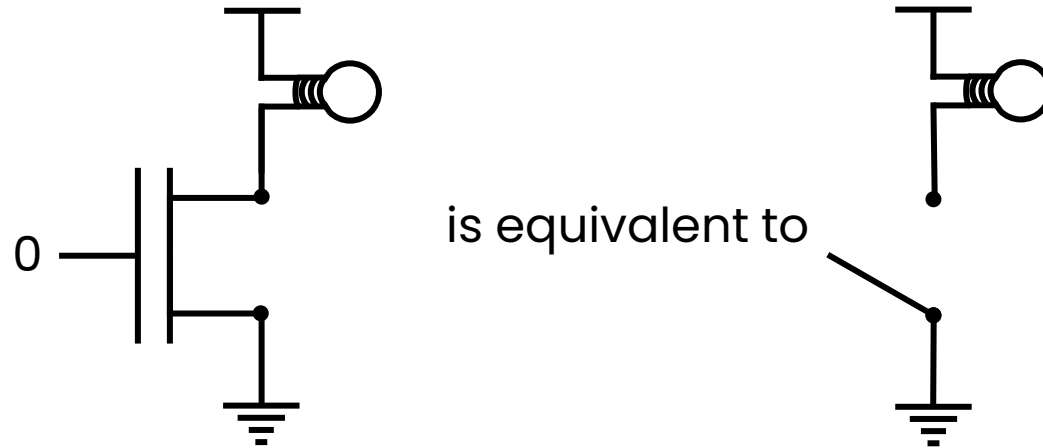
n-type



p-type

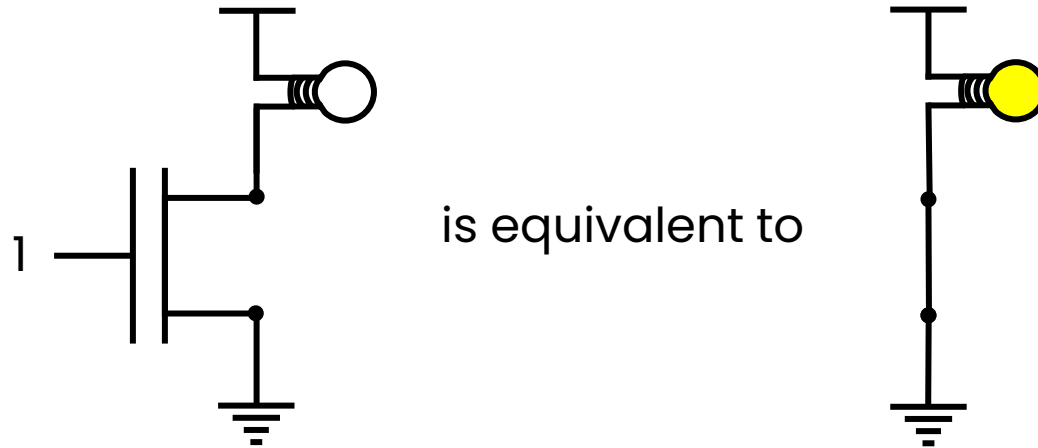
Transistor as a Switch

- MOSFET Switching Behavior
 - n-type



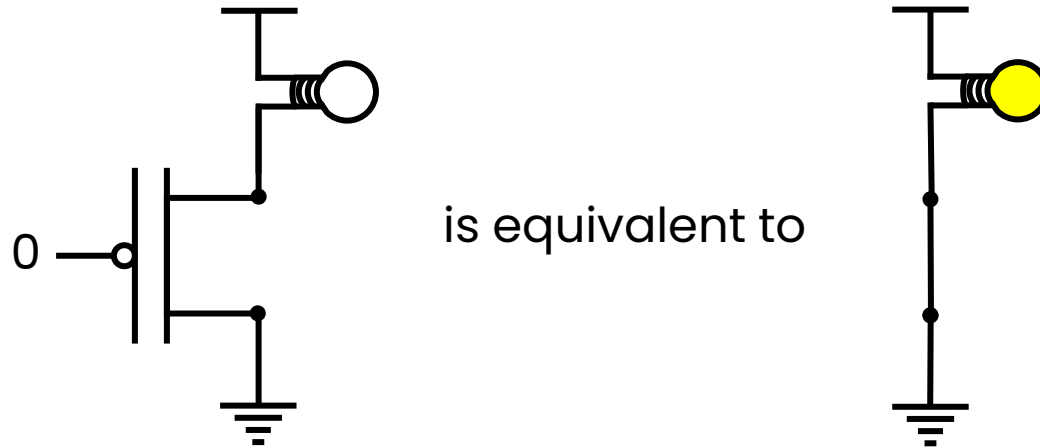
Transistor as a Switch

- MOSFET Switching Behavior
 - n-type



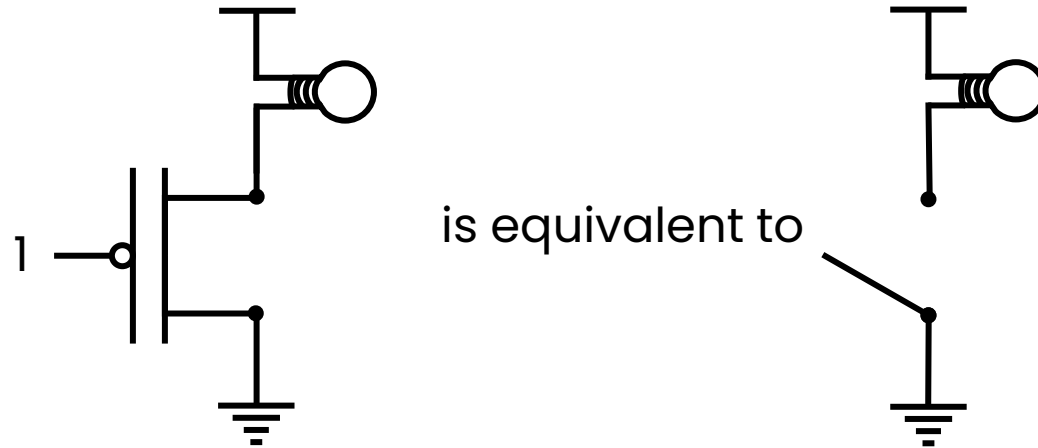
Transistor as a Switch

- MOSFET Switching Behavior
 - p-type



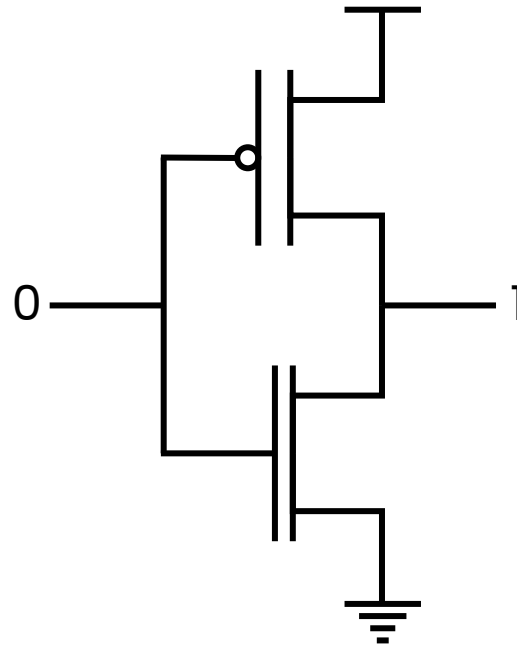
Transistor as a Switch

- MOSFET Switching Behavior
 - p-type



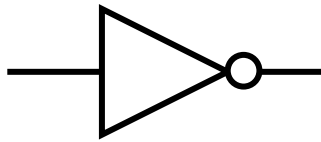
Building Logic Gates

- Inverter or NOT Gate

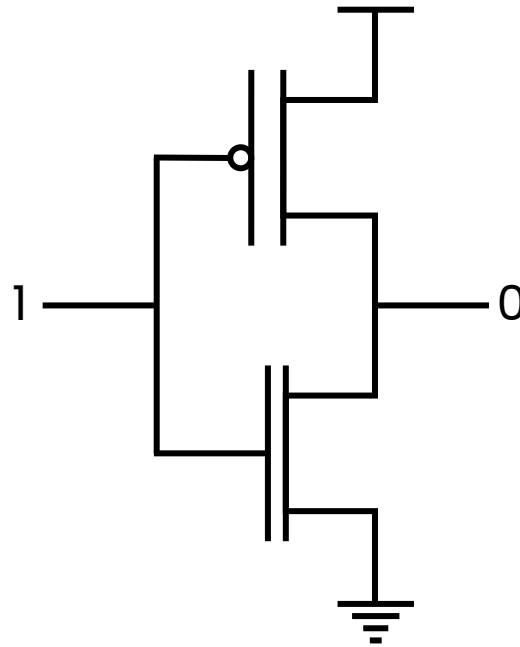


Building Logic Gates

- Inverter or NOT Gate

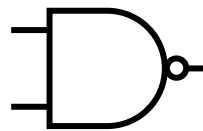


Symbol

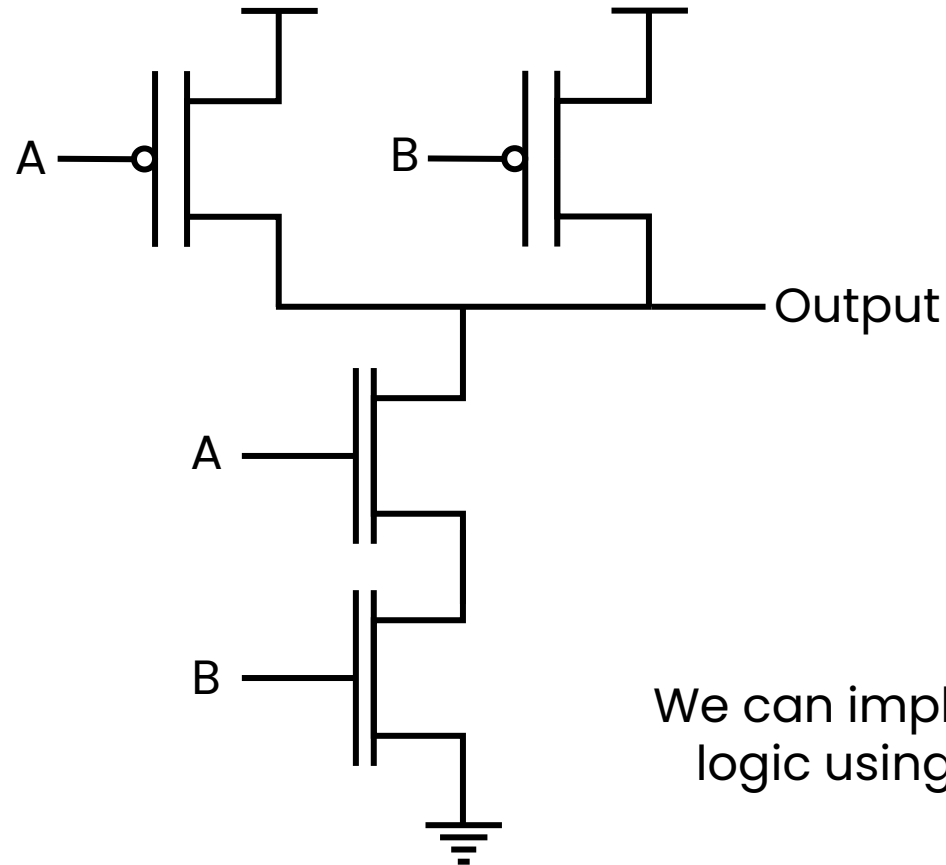


Building Logic Gates

- NAND Gate



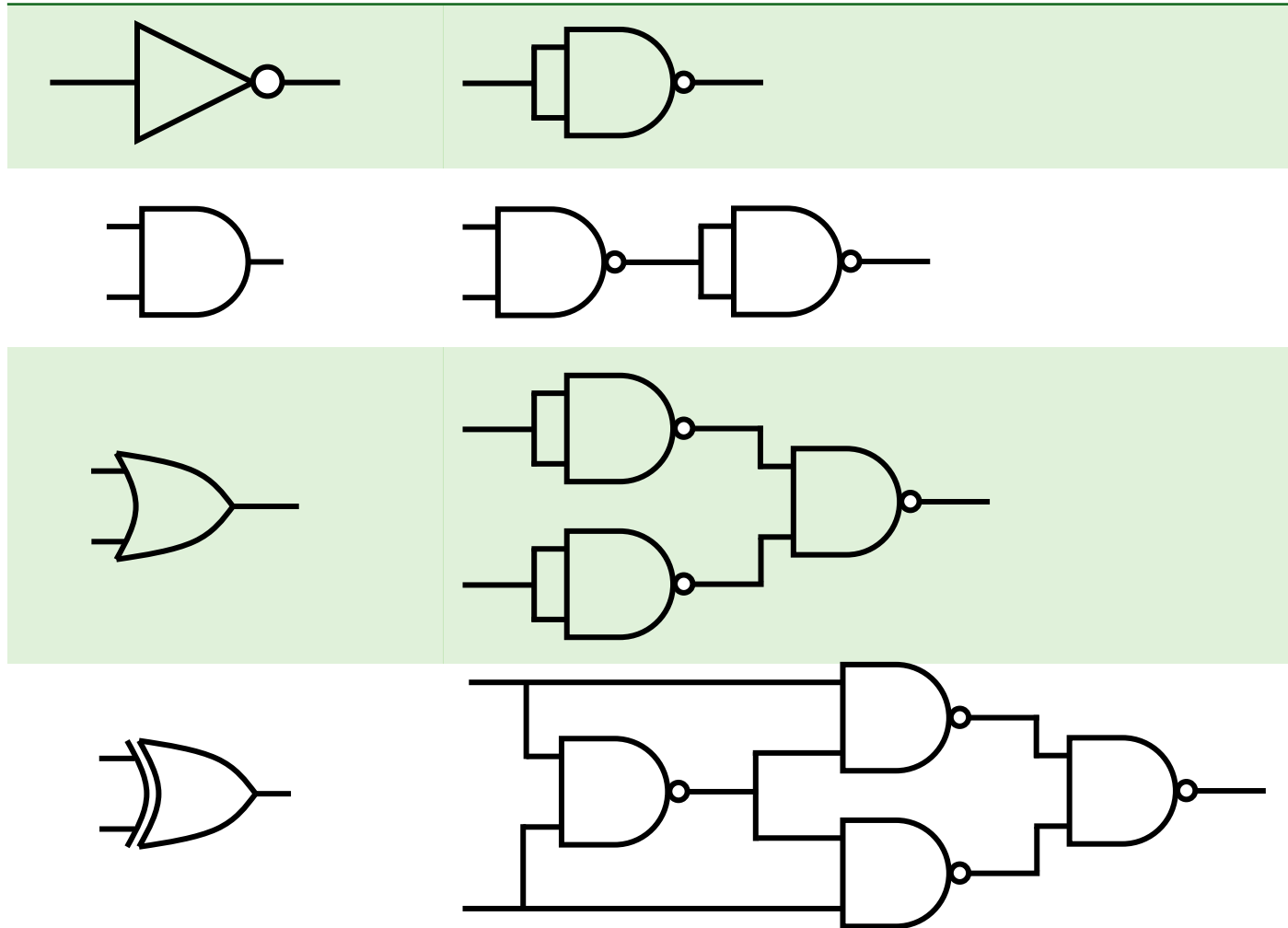
Symbol



We can implement any Boolean logic using only NAND gates!

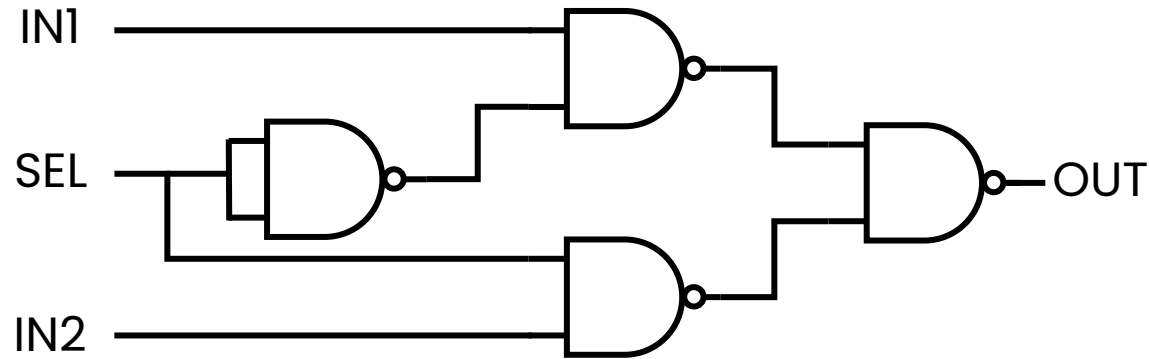
Building Logic Gates

- NAND (or NOR) Gate is universal gate

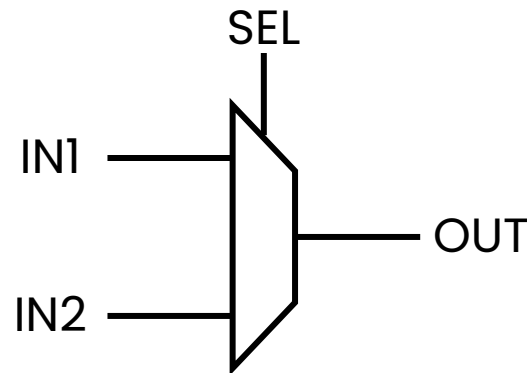


Combinational Logic

- What is this?



Two Input
Multiplexer!

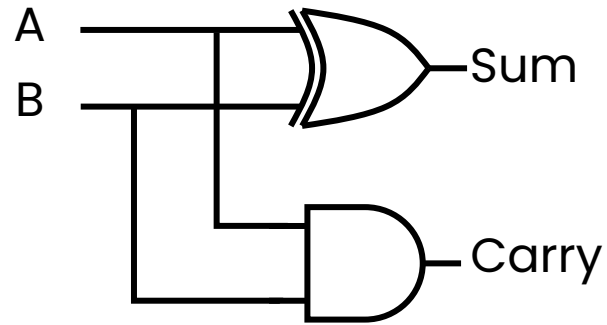


SEL	OUT
0	IN1
1	IN2

Combinational Logic

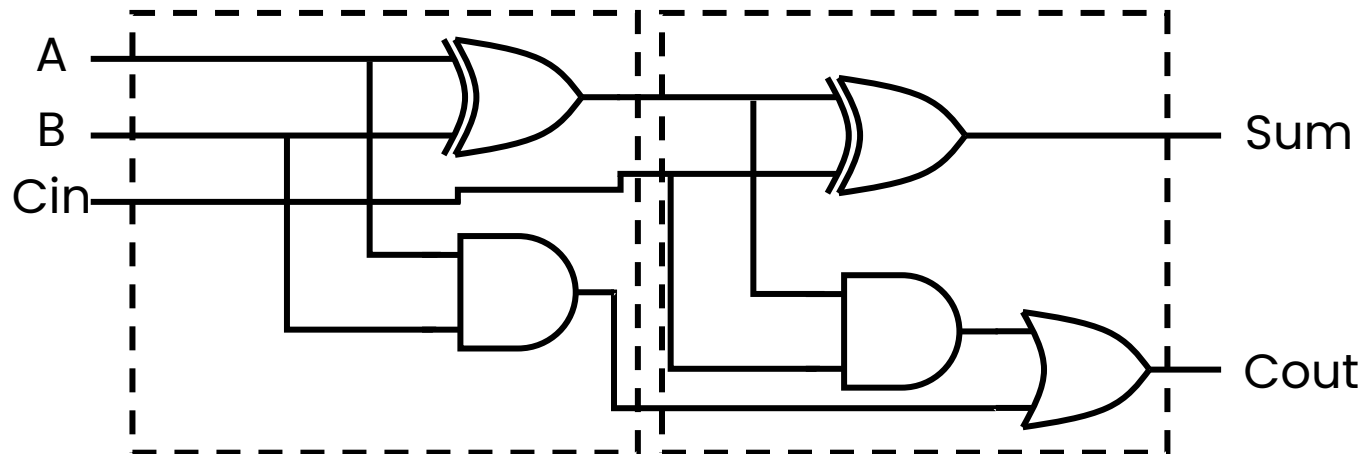
- What is this?

Half Adder!



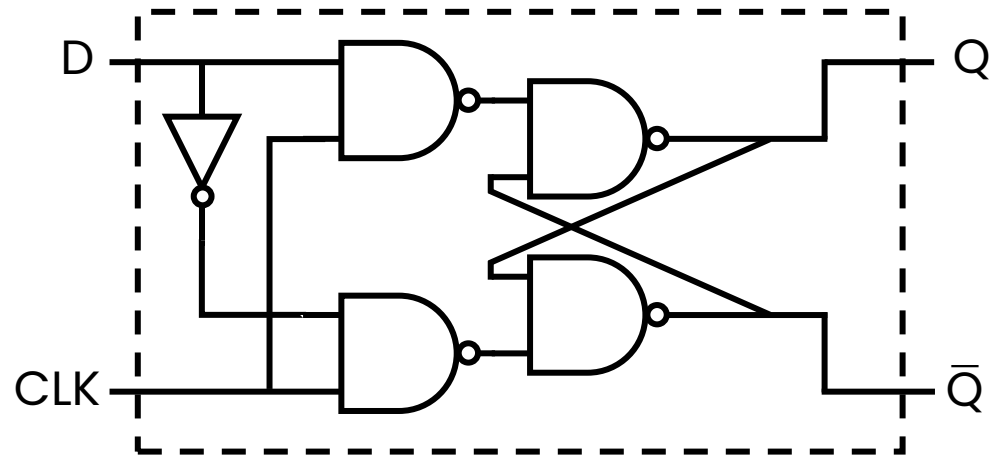
Can we make a Full Adder
from two Half Adders?

We can also implement
using only NAND Gates!

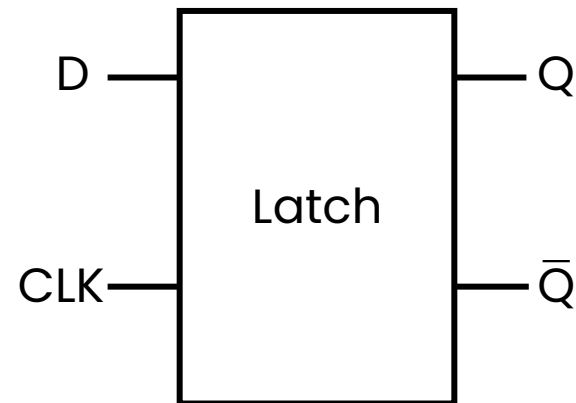


Sequential Logic

- Latch

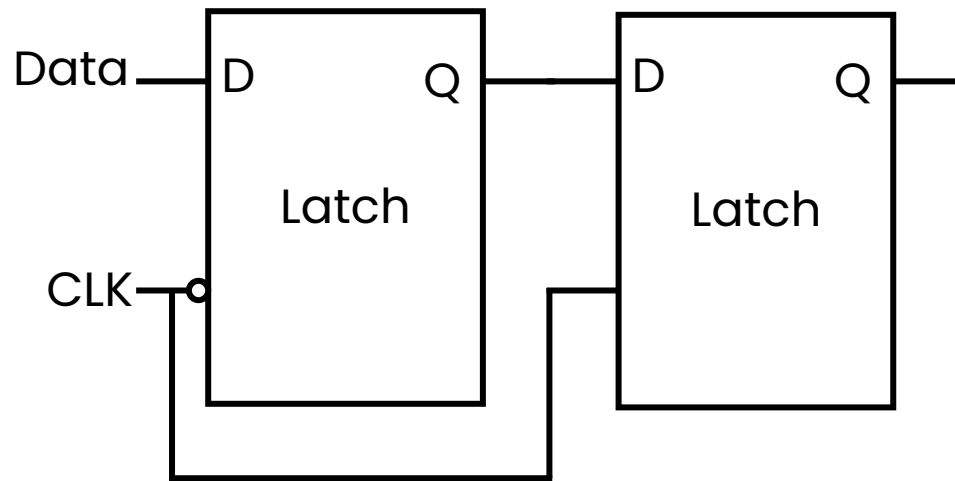


CLK	D	Q
1	0	0
1	1	1
Else	x	Q



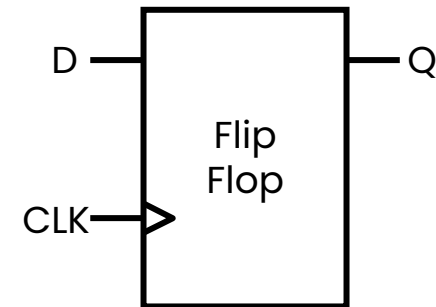
Sequential Logic

- Flip Flop / Register
 - Implemented using latches
 - There are other ways too ...



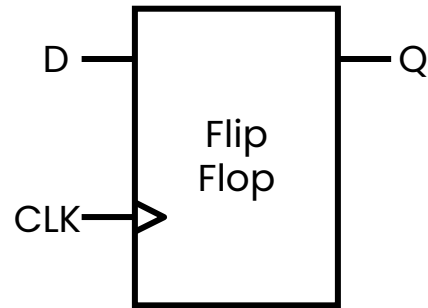
Multiple flip flops can be combined to store multiple bits!

CLK	D	Q
Rising	0	0
Rising	1	1
Else	x	Q

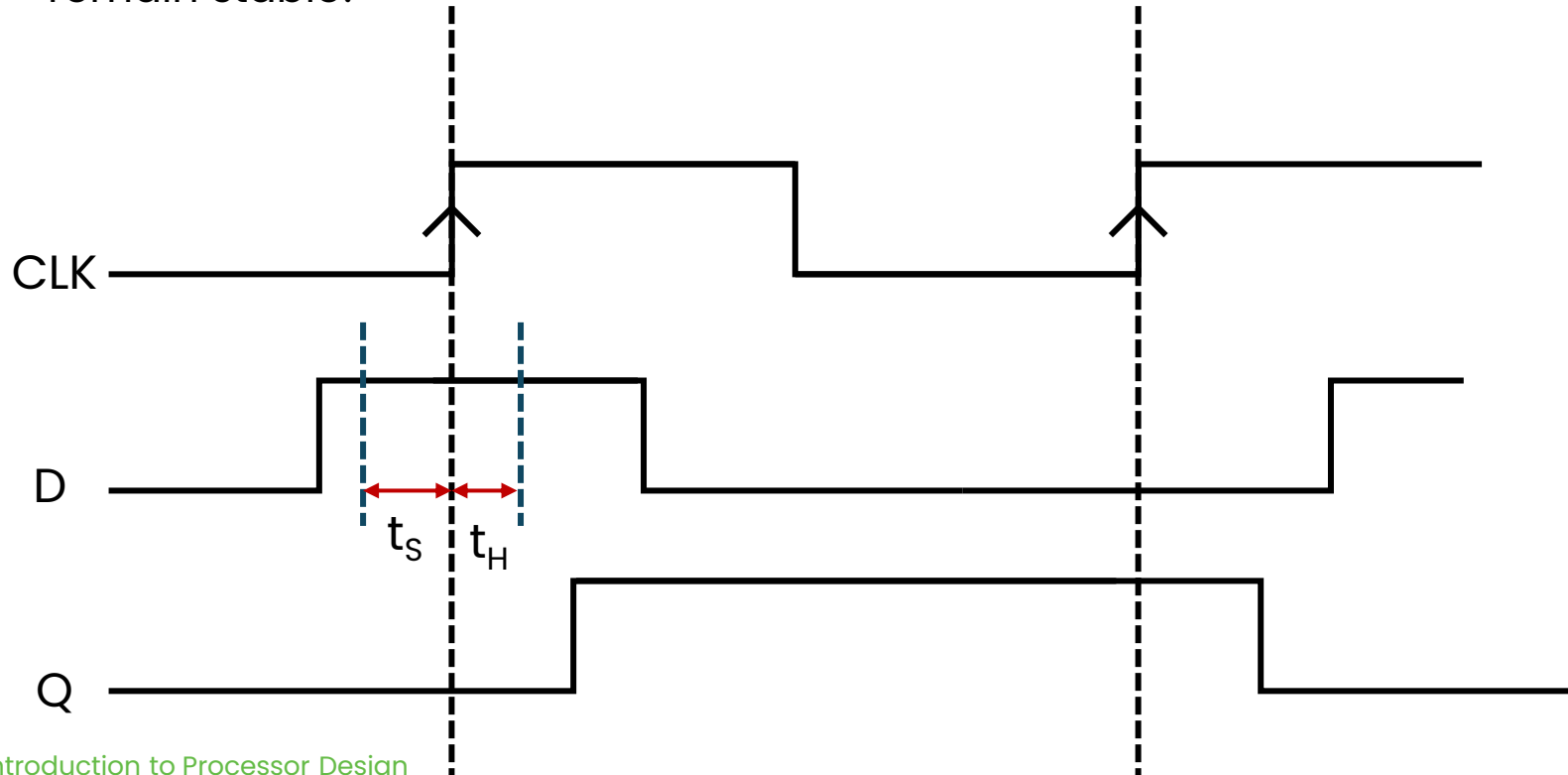


Timing in Sequential Circuits

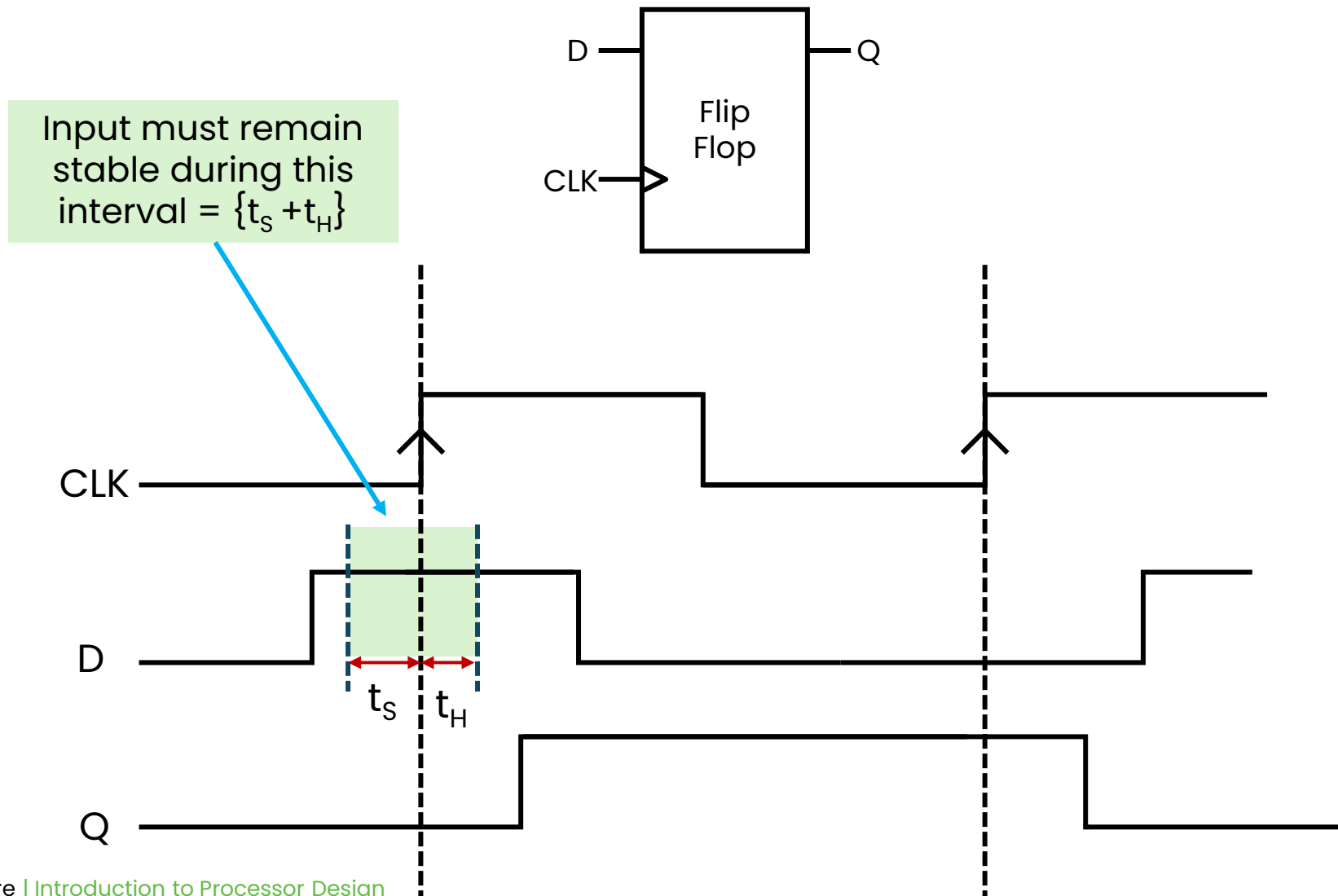
Setup Time (t_s)
Minimum Time before
clock edge during
which input must
remain stable!



Hold Time (t_H)
Minimum Time after
clock edge during
which input must
remain stable!

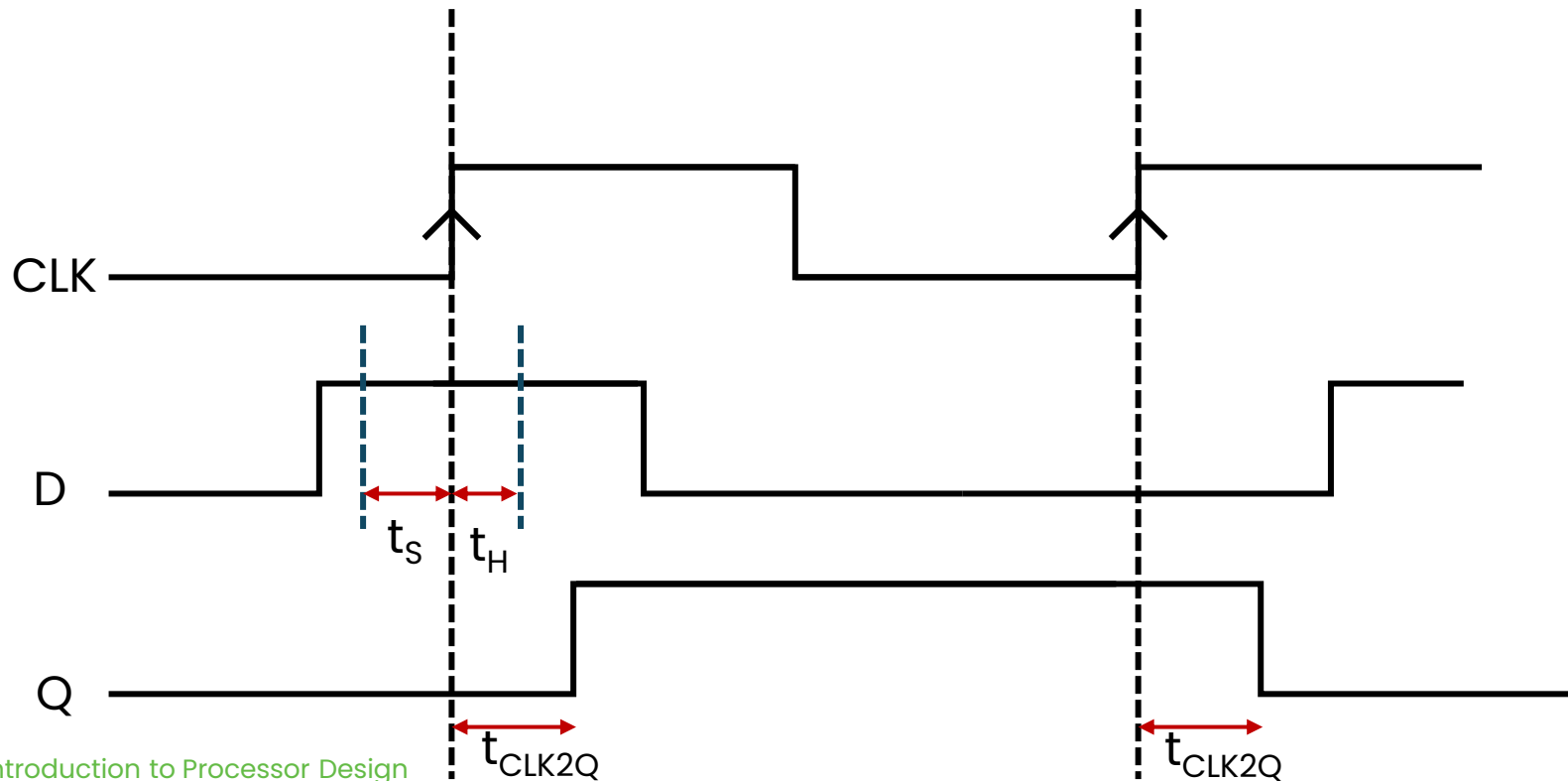
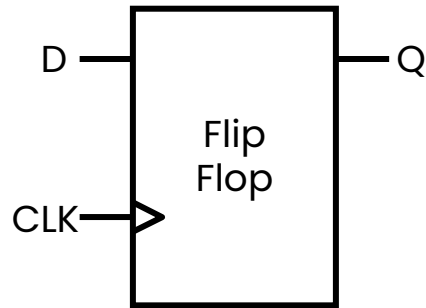


Timing in Sequential Circuits

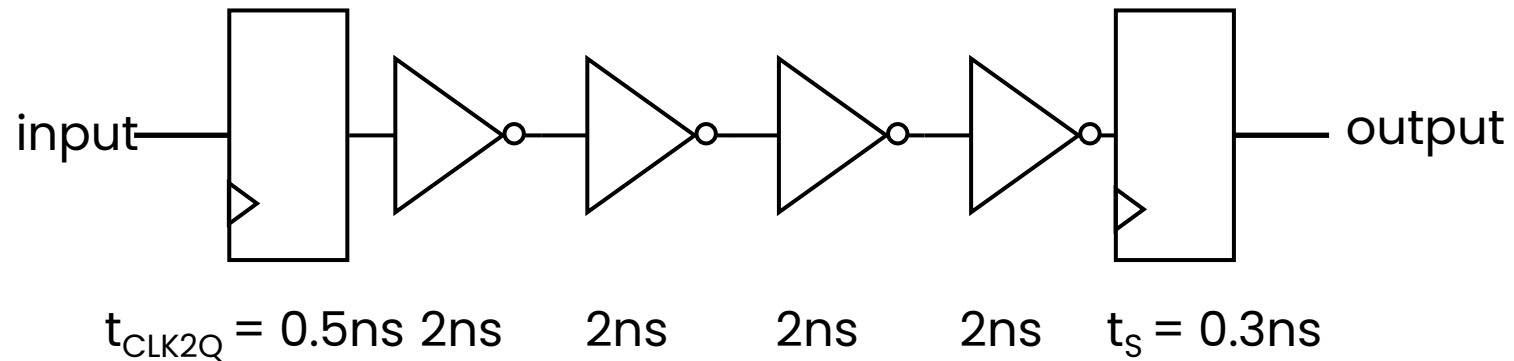


Timing in Sequential Circuits

Propagation Delay (t_{CLK2Q})
Time to propagate input to
the output after clock
edge!



Critical Path and Clock Rate



Minimum Clock Period = the critical path delay between two sequential elements!

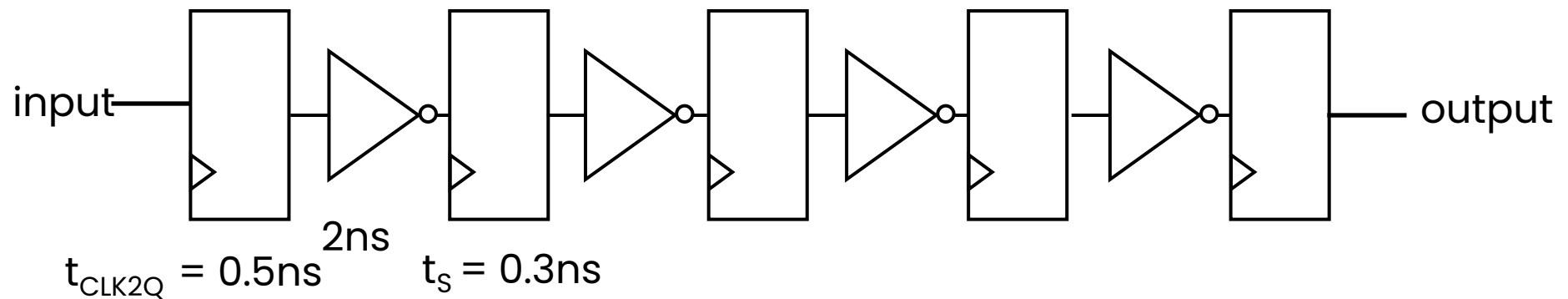
$$\text{Maximum Clock Frequency} = \frac{1}{\text{critical path delay}}$$

$$\text{Maximum Clock Frequency} = \frac{1}{t_{\text{CLK2Q}} \text{ of source register} + \text{combinational delay} + t_s \text{ of destination register}}$$

$$\text{Maximum Clock Frequency} = \frac{1}{0.5\text{ns} + 2\text{ns} + 2\text{ns} + 2\text{ns} + 2\text{ns} + 0.3\text{ns}} = \frac{1}{8.8\text{ns}} = 113.6 \text{ MHz}$$

Critical Path and Clock Rate

- Reducing the critical path to increase clock rate!

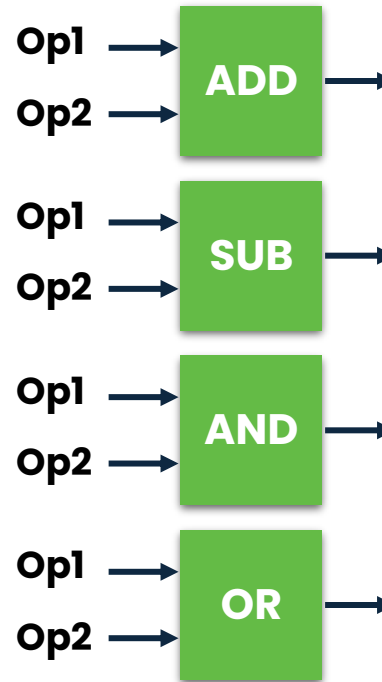


$$\text{Maximum Clock Frequency} = \frac{1}{0.5ns + 2ns + 0.3ns} = \frac{1}{2.8ns} = 357.14 \text{ MHz}$$

**We have got all the ingredients to build a
simple **ALU** or even a simple
Microprocessor ...**

Building a Simple ALU

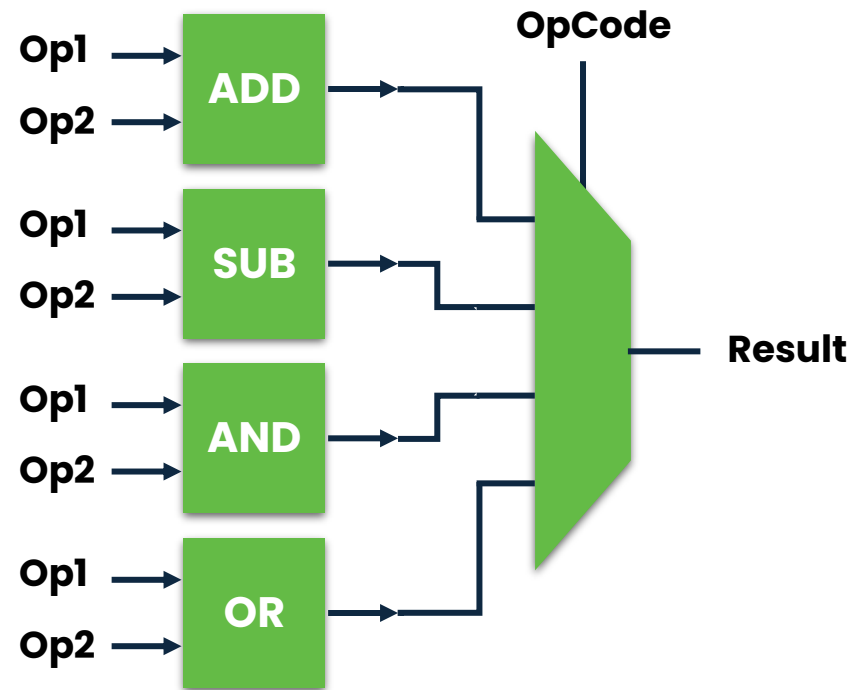
- Let's build an ALU with two operands and four operations!



The operands can be single-bit or multi-bit!

From ALU to Microprocessor

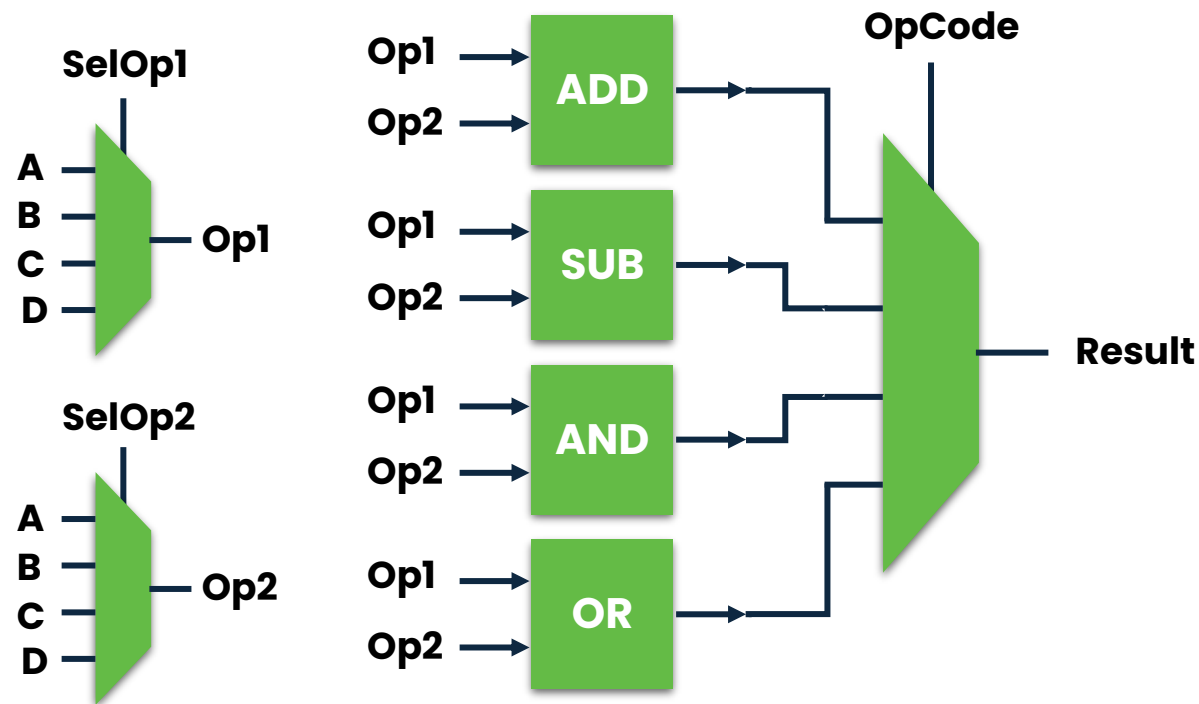
- Let's add logic to select result



2-bit OpCode selects one of the four operations!

From ALU to Microprocessor

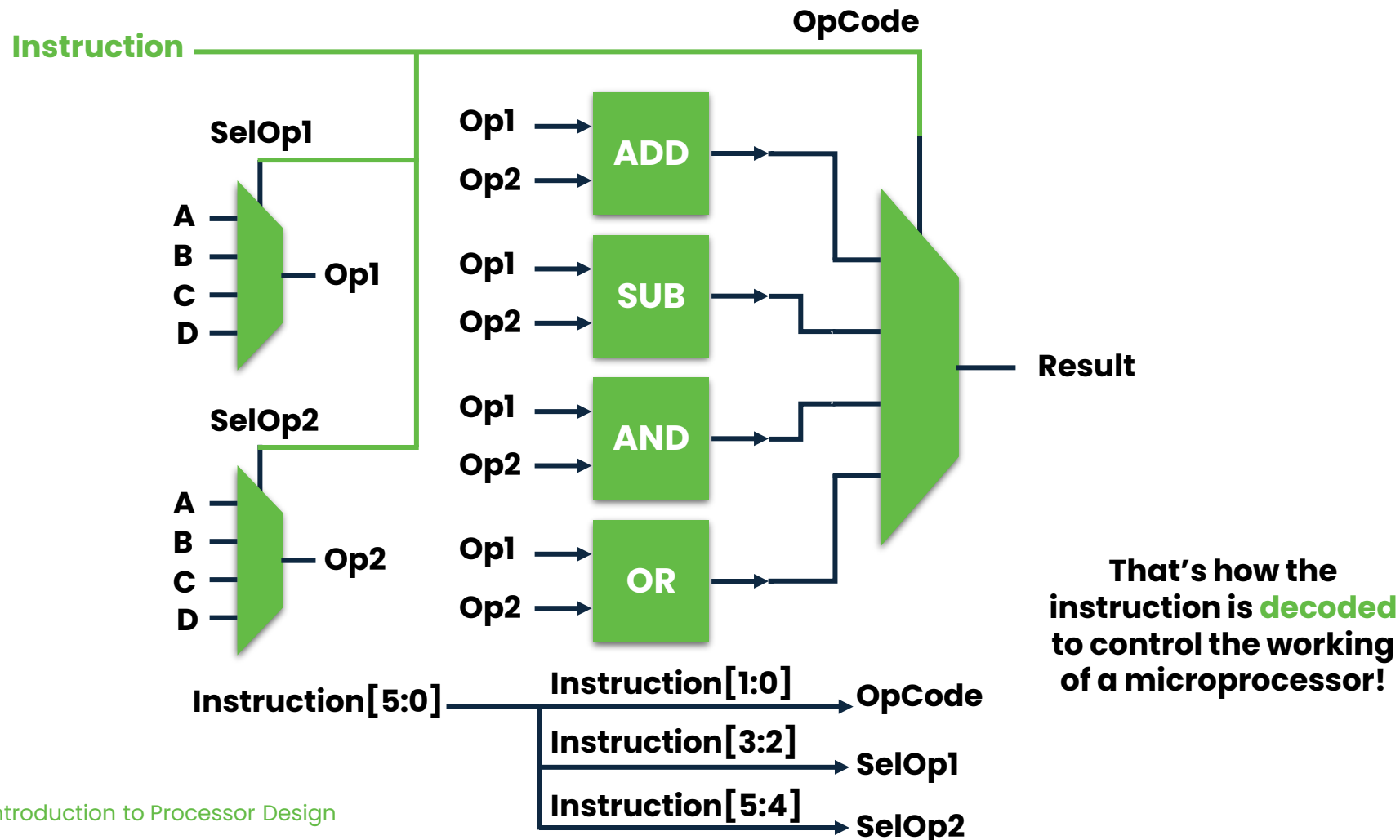
- Selecting two operands from four possible ones!



Together **SelOp1**, **SelOp2** and **OpCode** form 6 bits that control the working of this simple Microprocessor!

A Simple Microprocessor

- This Microprocessor operates using **6-bit Instruction!**



Programming Microprocessor

- To program simple microprocessor, we can use 6-bit instruction
 - Suppose we want to add B and C
 - OpCode = 00 for ADD
 - SelOp1 = 01 to select B
 - SelOp2 = 10 to select C
 - Corresponding Instruction is 100100 i.e., {SelOp2, SelOp1, OpCode}!
 - 100100 is **machine code** or **machine language**
 - That's the lowest level **programming language** understood directly by the hardware!
 - That's not quite readable or easier to use for humans, isn't it?
 - So, we use high-level languages!

Programming Microprocessor

- **ADD B, C** could be a possible syntax of assembly language instruction for our simple microprocessor!
 - Similarly, we can have **AND A, D**
 - **OR C, A** and so on ...
 - All the possible instructions form the **instruction set** for our microprocessor!

Programming Microprocessor

- High-level programming languages simplify our task beyond assembly language!
- For instance, a statement such **Result = A+B+C+D** in C language may **replace** following assembly language instructions
 - ADD B, C
 - STORE Temp1, Result
 - ADD C, D
 - STORE Temp2, Result
 - ADD Temp1, Temp2
 - We will need to add some more logic and temporary registers in our microprocessor to support all the above instructions!

Programming **Microprocessor**

- **Compiler**

- Translates software from a high-level language to assembly language!
- There can be different assembly language codes for the same high-level language code!

- **Assembler**

- Translates the assembly language to machine language!
- There is exactly one possible machine instruction for every assembly language instruction!

- **Microarchitecture**

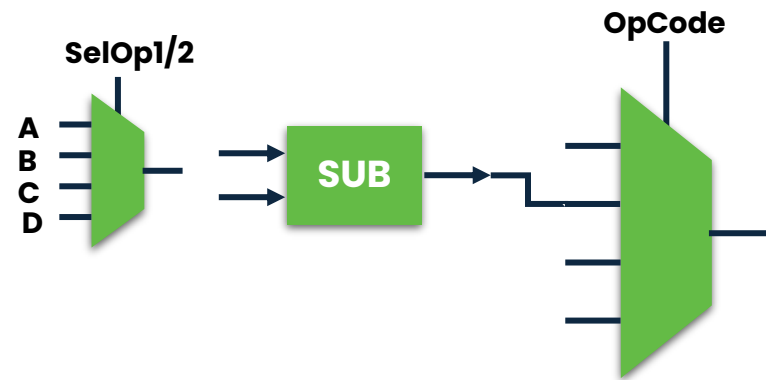
- The logic that executes the machine language!

Measuring Performance

- The implementation is simple!
 - Pure combinational circuit!
 - Output changes when input instruction changes!
- Performance
 - If we add a register at input (for instruction) and one at output (for result), each instruction takes one cycle to execute!
 - CPI (cycles per instruction) = 1
 - IPC (instructions per cycle) = 1

Measuring Performance

- What's the clock rate of this processor?
 - Depends on the critical path!
 - Critical path consists of
 - Two multiplexers
 - Subtractor (longest path in the ALU blocks!)



$$\bullet \text{ Max.Clock Rate} = \frac{1}{2 \times (\text{Multiplexer Delay}) + \text{Subtractor Delay} + t_S + t_{CLK2Q}}$$

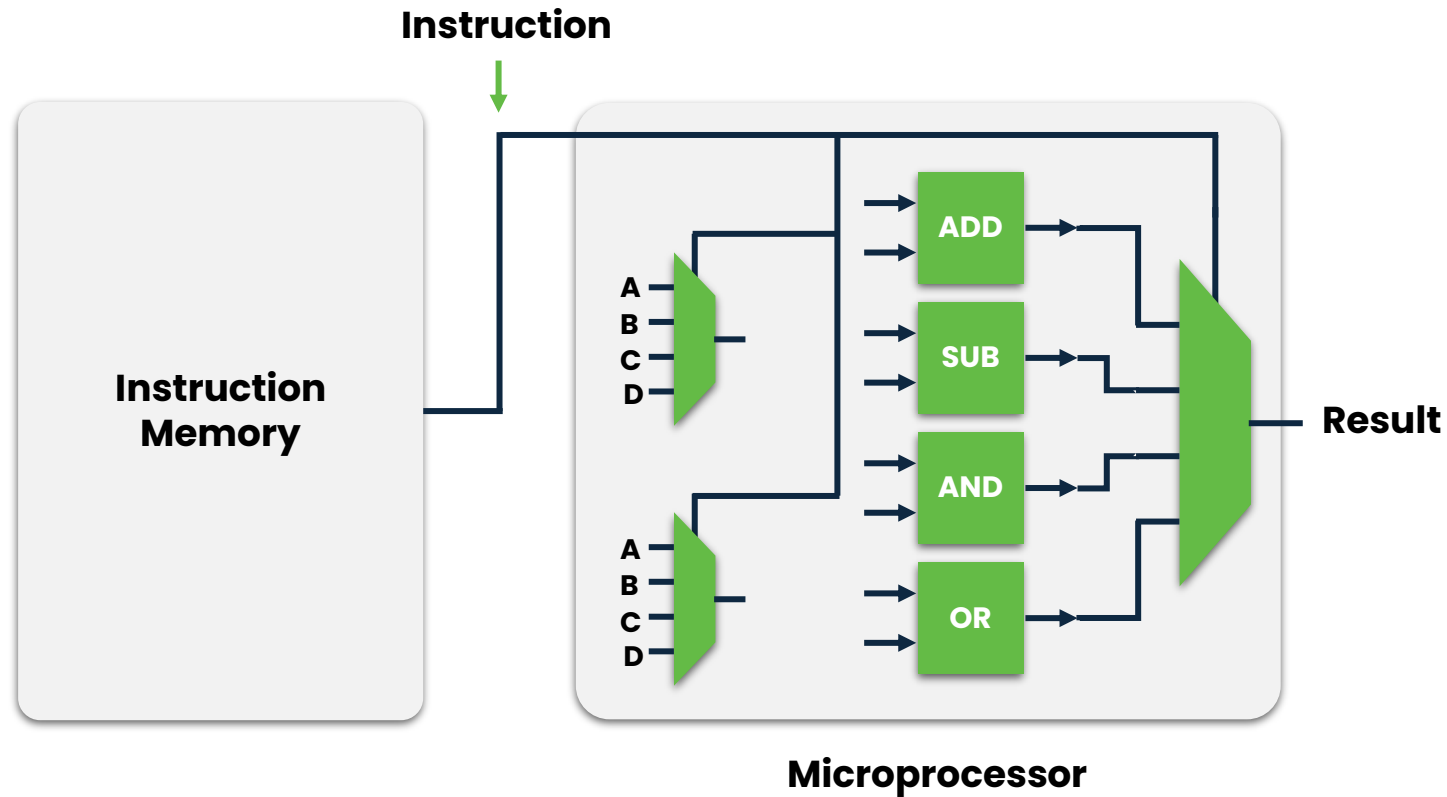
Want to execute **multiple instructions?**



Let's add **memory** for multiple instructions

Adding Instruction Memory

- **Instruction Memory** can hold multiple instruction!

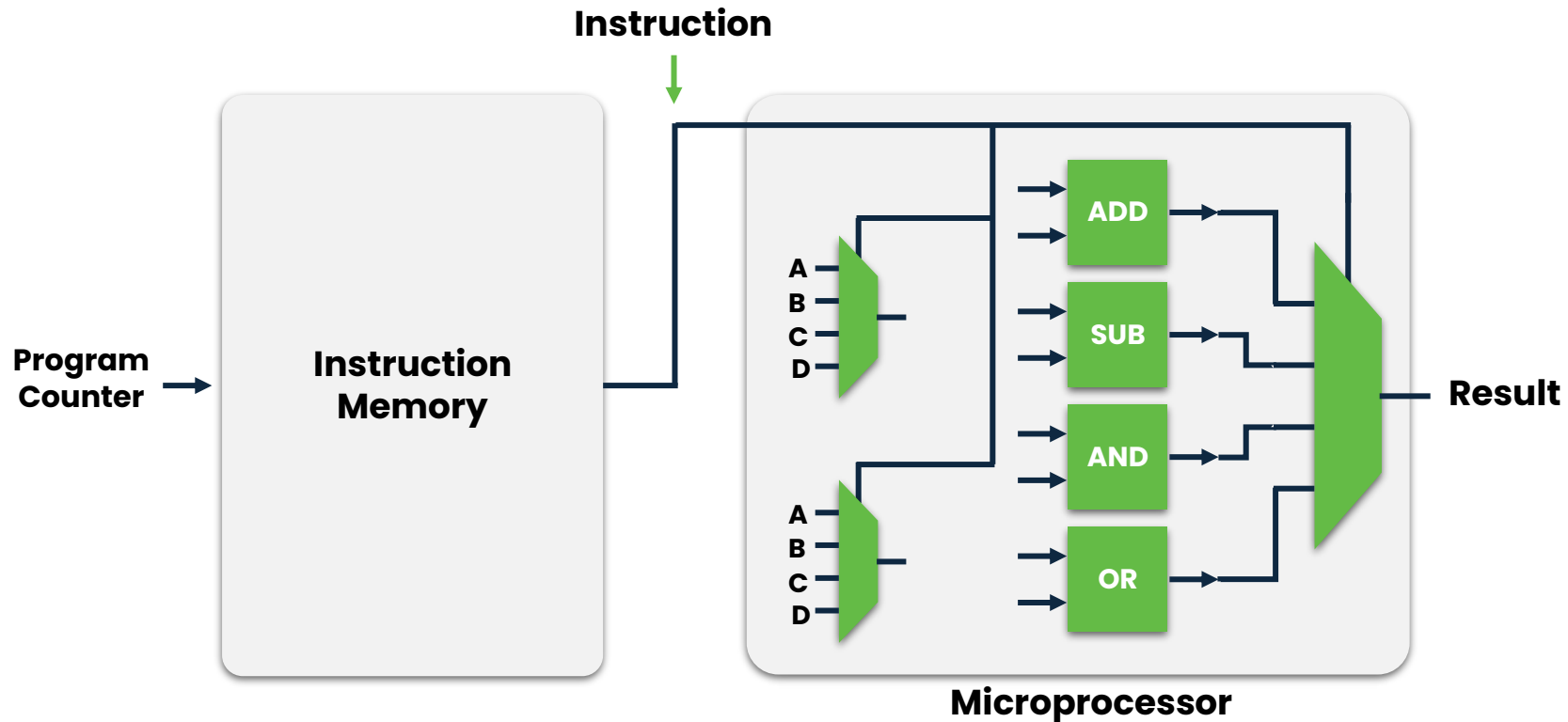


Adding Instruction Memory

- 6-bit Instructions require a simple memory where each location may be 6-bit
 - Practical memories are sized in bytes (8 bits) and usually an address corresponds to a byte address!
- Let's add an instruction memory of size **8x6** bits
 - 8 locations
 - Each location is 6 bits in size!
 - How many bits do we need for address?
 - 3 bits
 - Instruction Cache is addressed using "**Program Counter**"
 - Also called "Instruction Pointer Register"!
 - Usually a *register* in actual microprocessors!

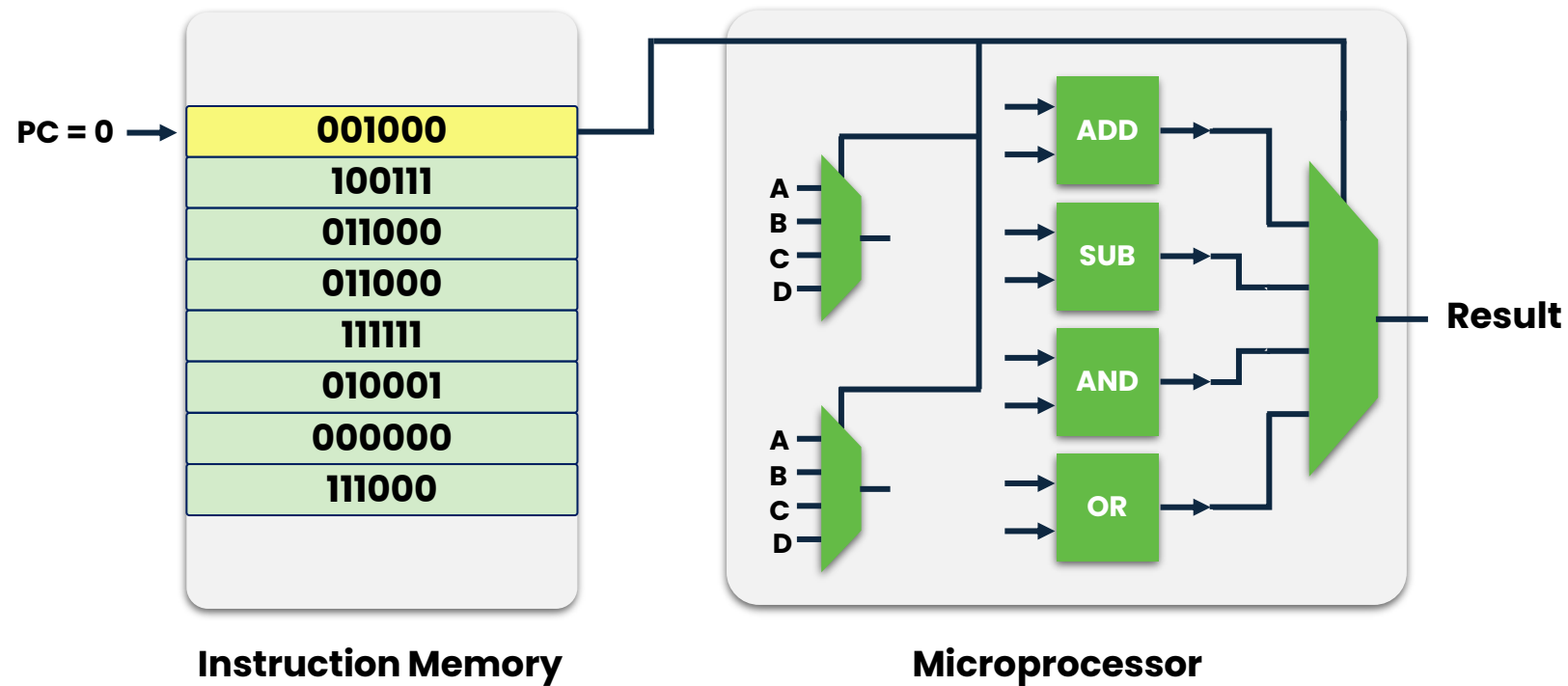
Program Counter / Instruction Pointer

- Program Counter (PC)
 - To address instruction memory!
 - For 8x6 bit instruction memory, PC is 3-bit!
 - PC can go from 000 to 111 to select 8 instructions!



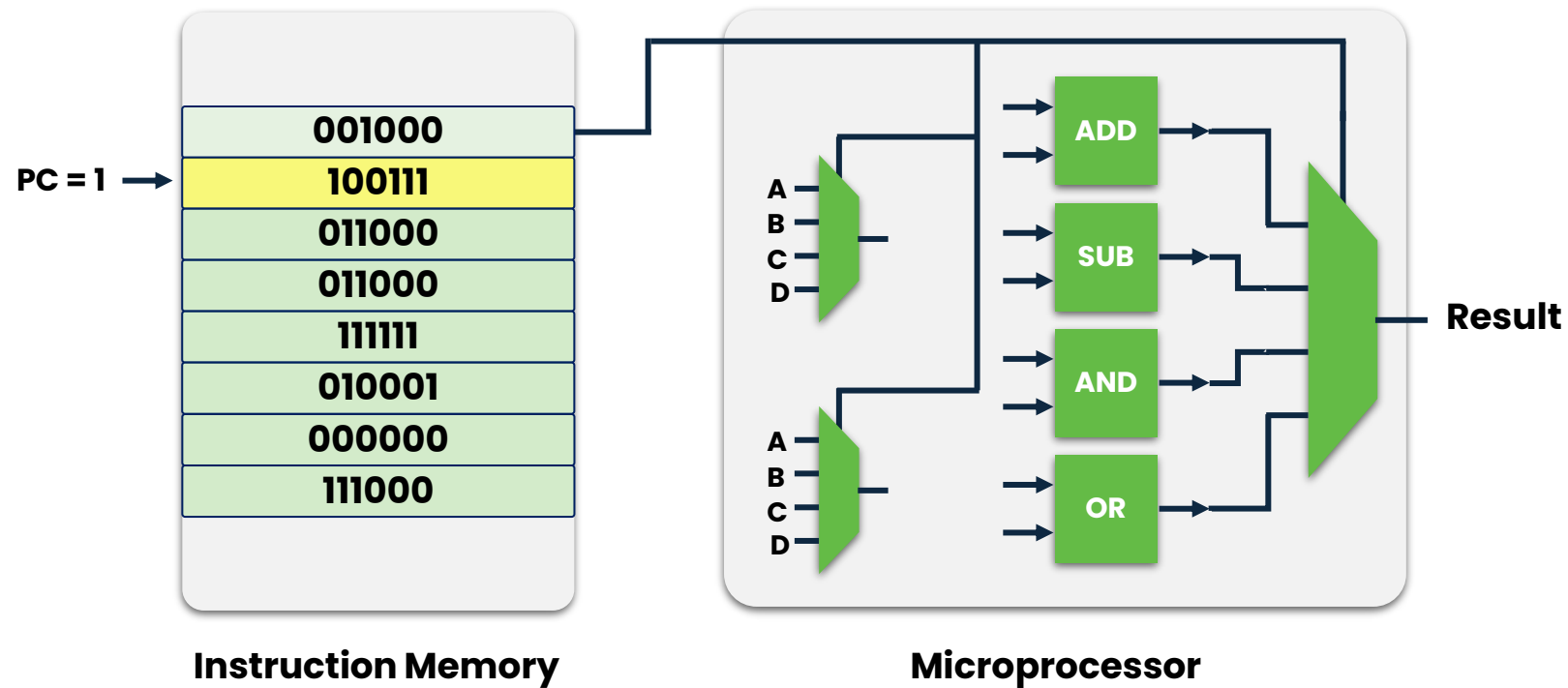
Executing Multiple Instructions

- PC is incremented to select successive instructions!



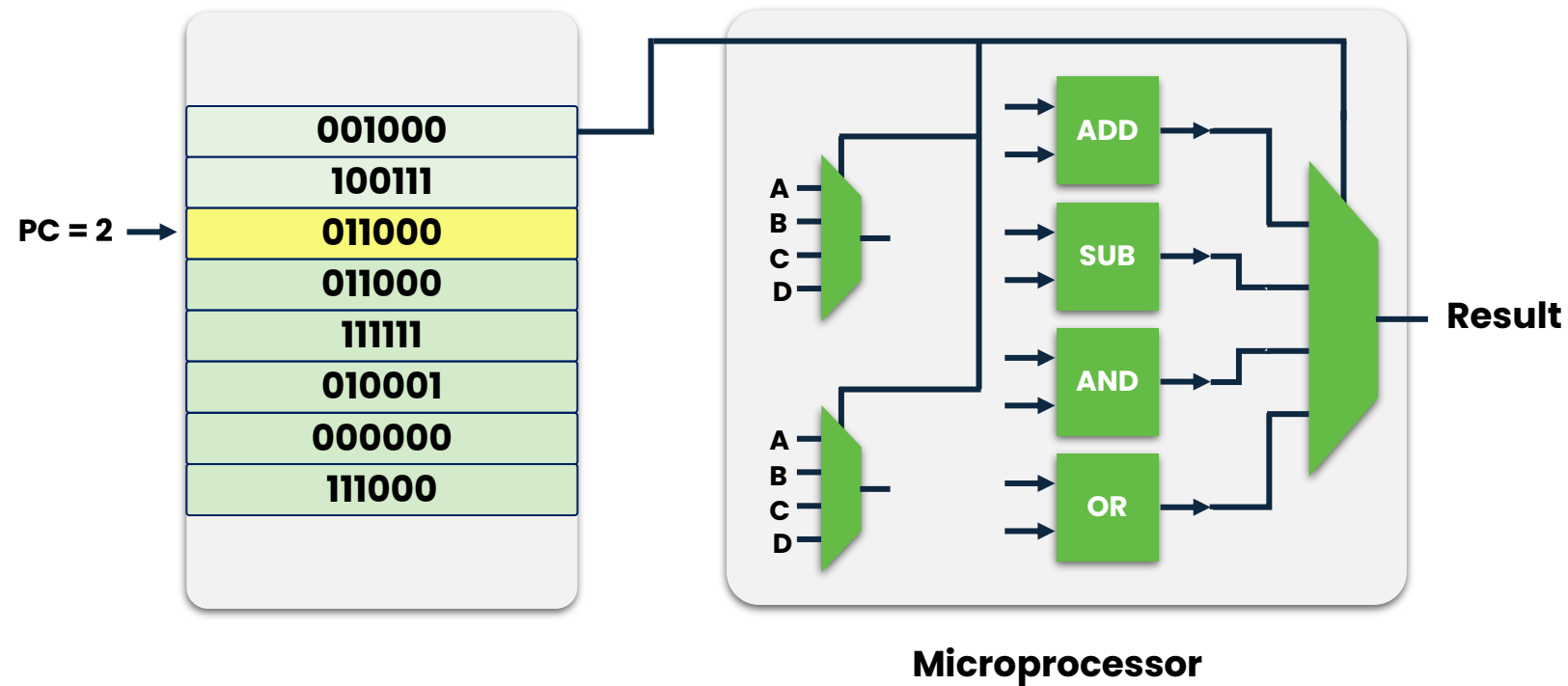
Executing Multiple Instructions

- PC is incremented to select successive instructions!



Executing Multiple Instructions

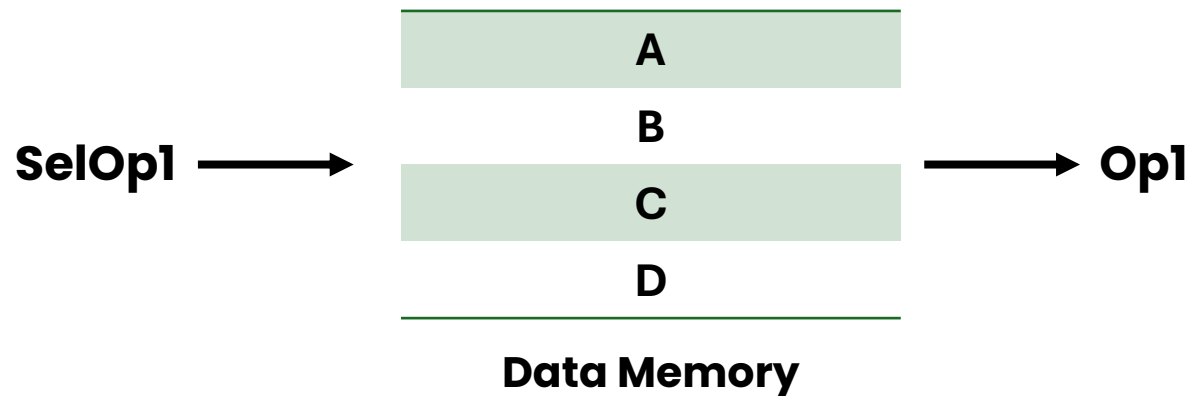
- PC is **incremented** to select successive instructions!



**Just like *Instruction Memory*, we have
Data Memory ...**

Adding Data Memory

- Can add two simple data memories to load operands! (multiplexers will be removed)
 - Each data memory can be $(4 \times n)$ bits
 - 4 locations
 - Each having an n -bit operand!
 - 2-bit address to select one of the four operands!



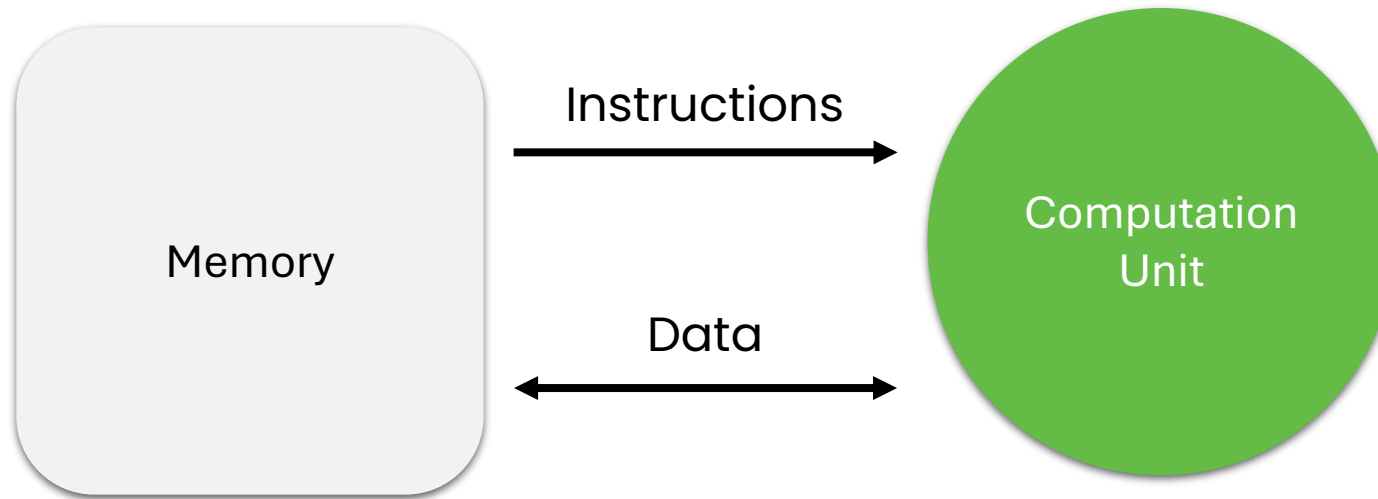
SelOp1 acts as *address*
of a variable in
memory!

Usually, **microprocessors**
load **one** operand at a
time from **memory**
instead of two!

Operands are loaded
into **registers** inside
CPU!

Processor Design – The Big Picture

- Mainly two components
 - **Memory** – Holds data and instructions
 - **The Computation Unit** – Performs Computation



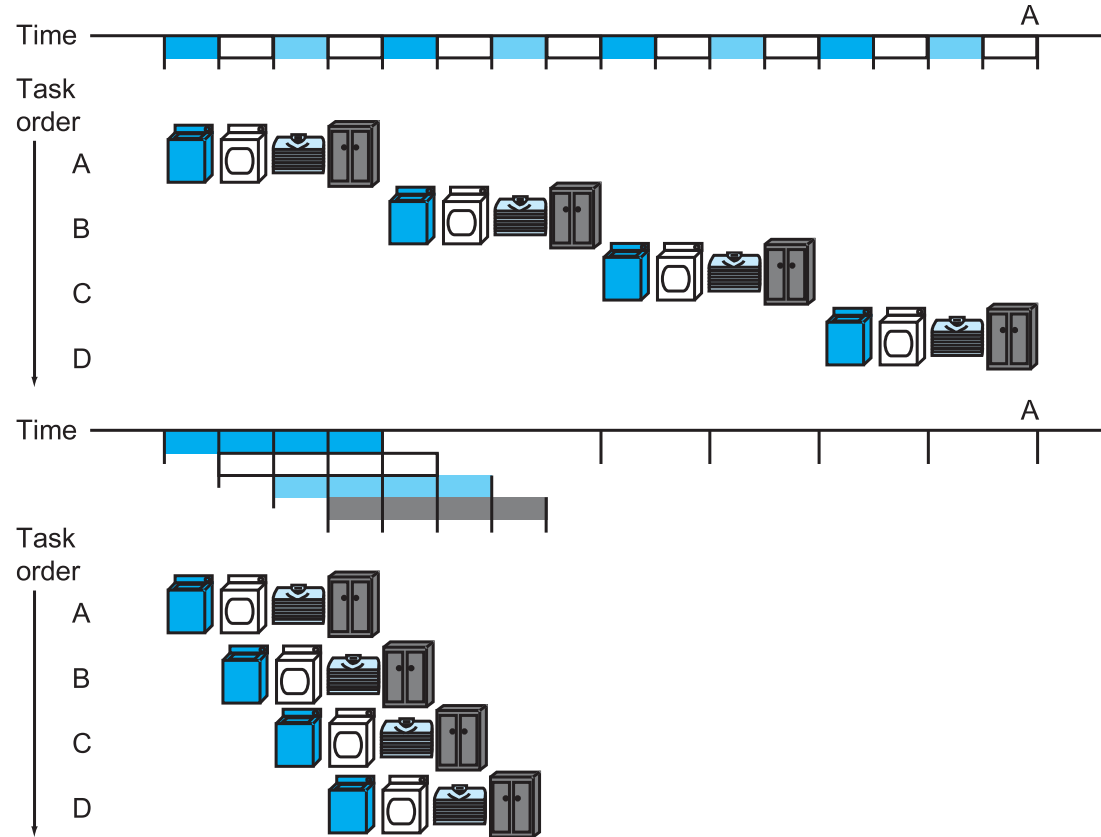
Can we **improve** our processor's
performance?



Why not!
Let's pipeline our processor design ...



A Laundry Analogy

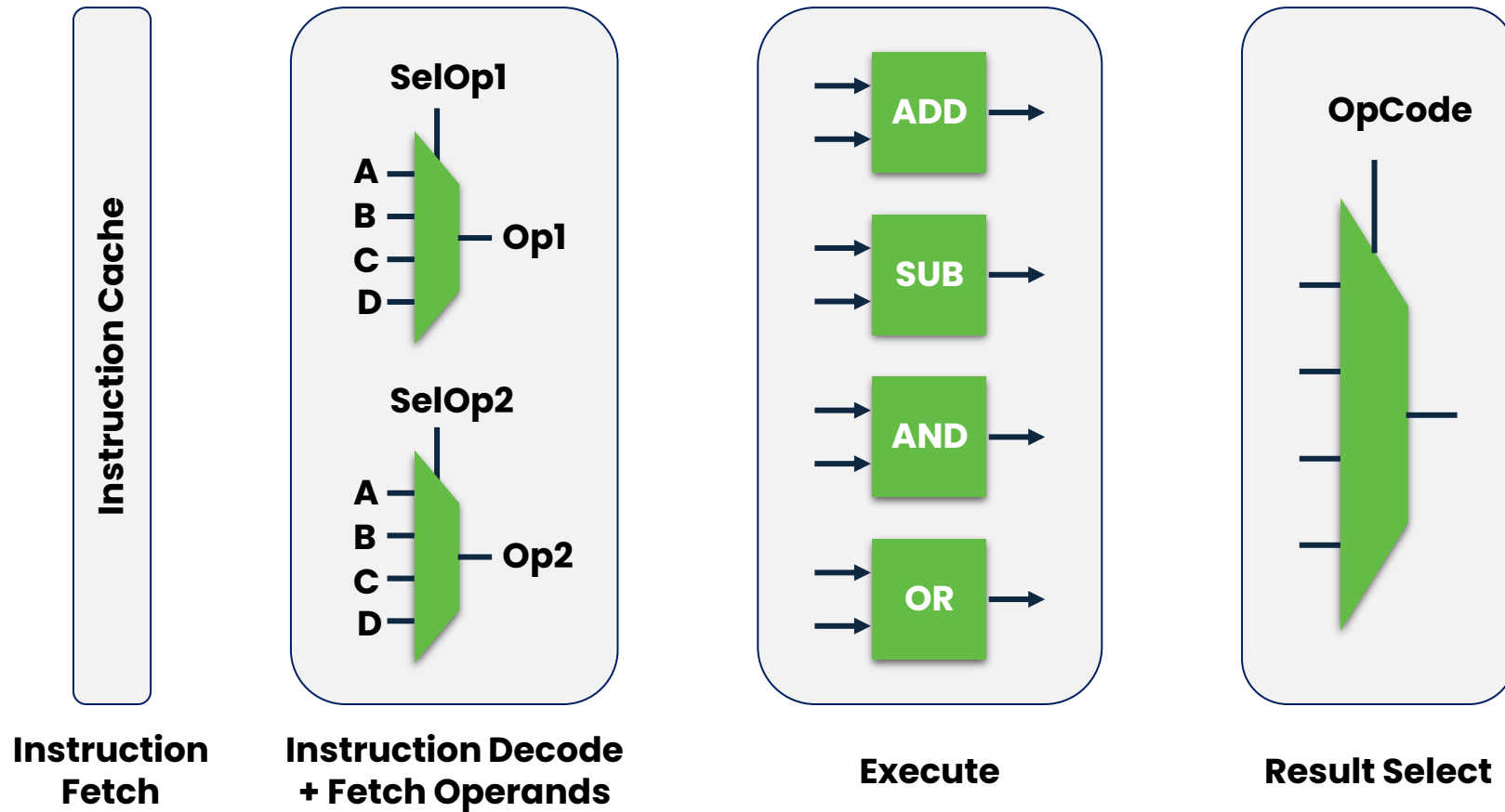


- Pipelining helps execute multiple tasks in parallel
- Improves throughput ...

Source: Computer Organization and Design (RISC-V Edition), Patterson and Hennessy

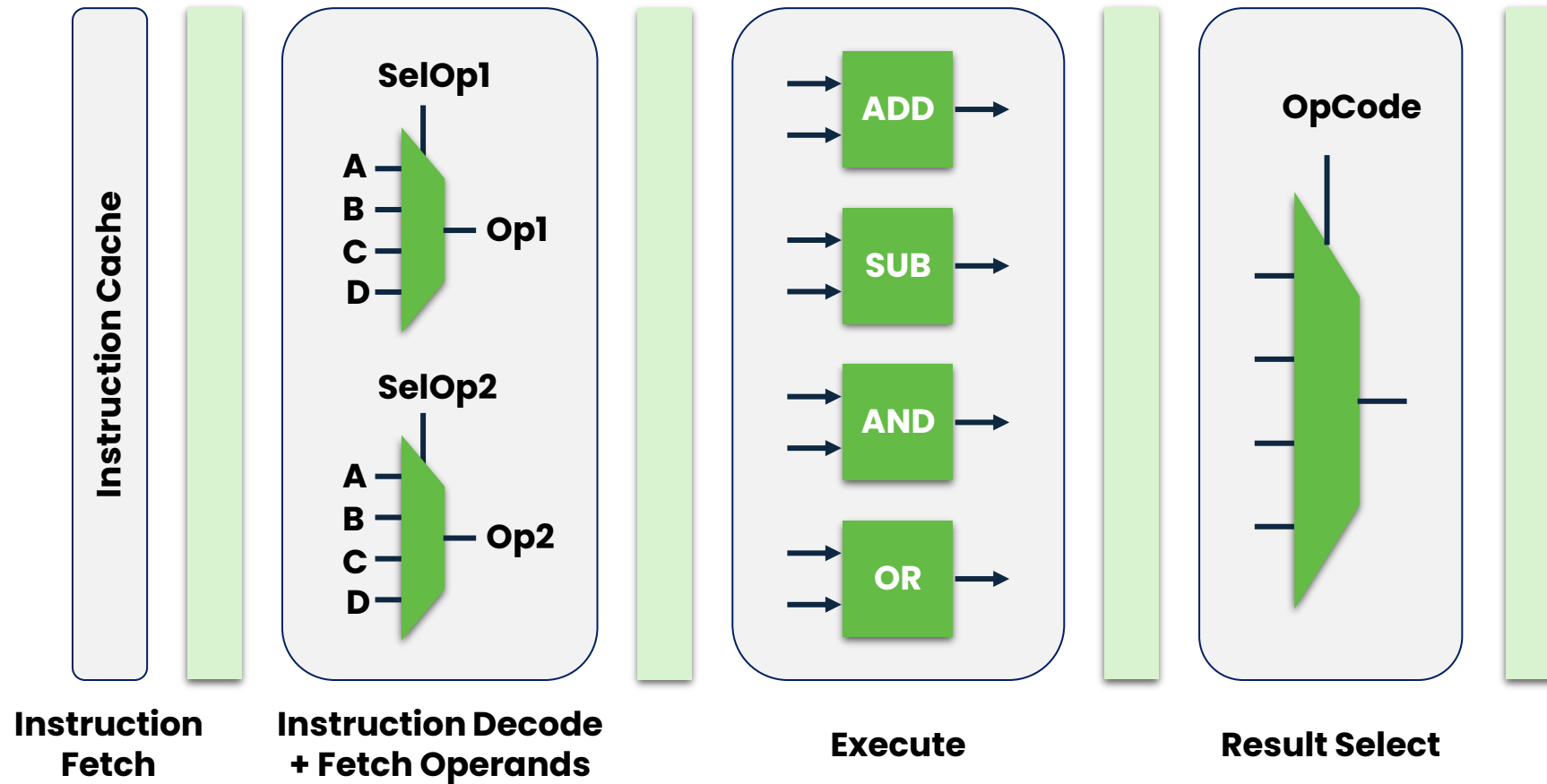
Pipelining Processor

- Without Pipelining



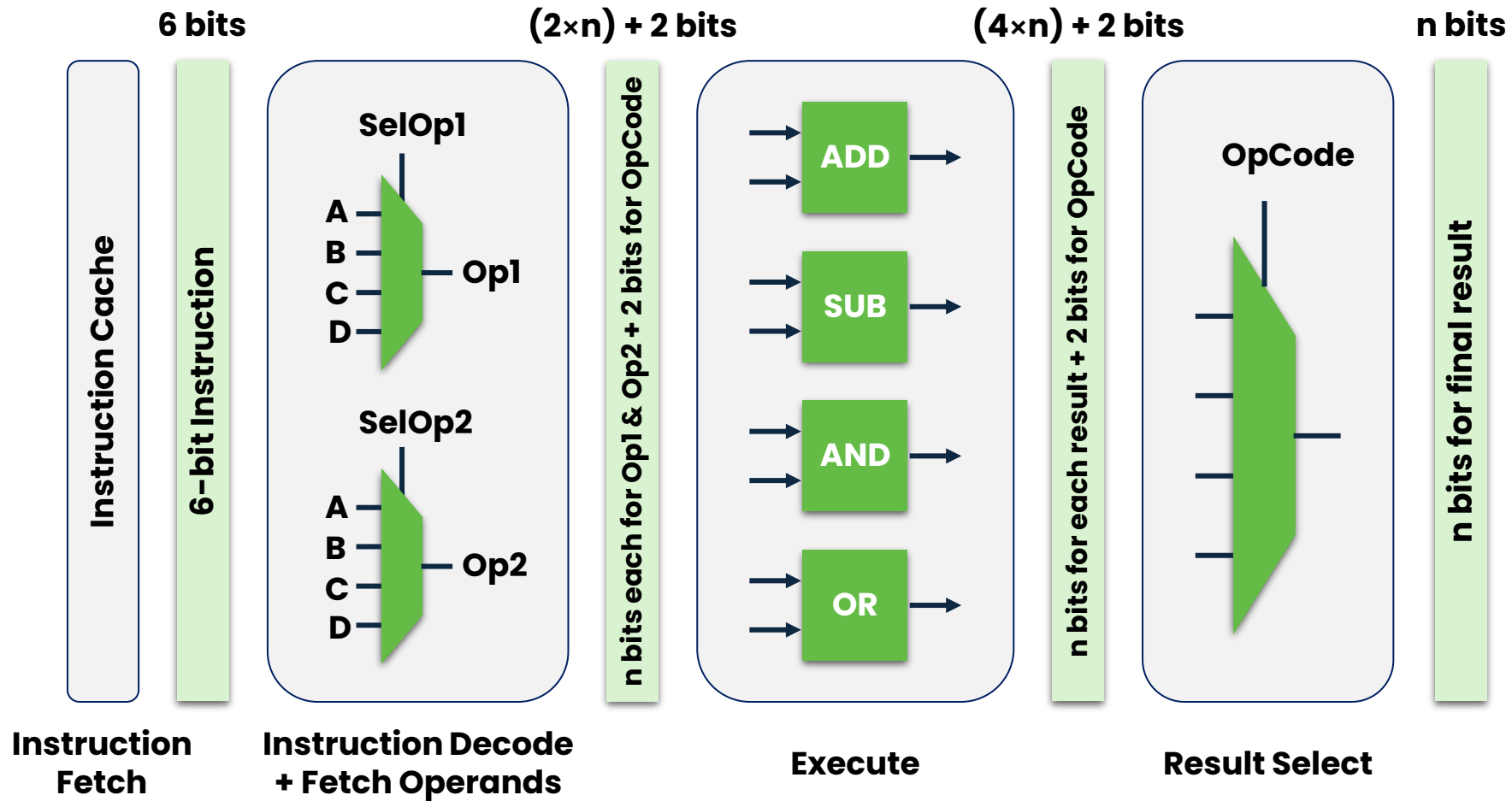
Pipelining Processor

- Adding Pipeline Registers



Pipelining Processor

- Register Sizes



What's the critical path in pipelined implementation?

Critical Path in Pipelined Design

- It's reduced to that of subtractor
 - Assuming subtractor path is longer than that of multiplexer!
- $Max\ Clock = \frac{1}{Subtractor\ Delay + t_S + t_{CLK2Q}}$
- Much faster than the previous design!
- As first instruction result (OpCode+Operands) goes to second pipelined register
 - A new instruction can be loaded from instruction cache to the first pipelined register!

Register **File**



In **load-store architectures** (all recent ones), data is **loaded** into **registers** inside CPU, which act as temporary storage, and after processing it is **stored** back in data **memory...**

Let's add a **Register File to our processor and modify the instruction set a bit ...**

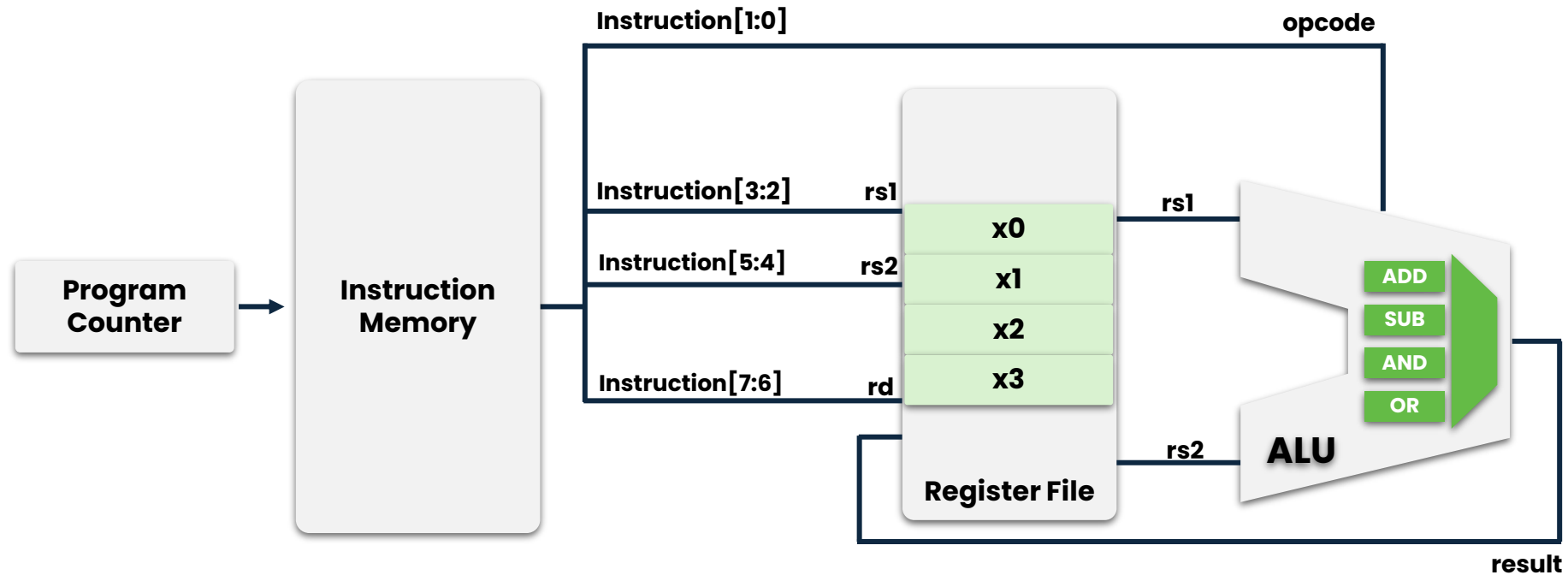
Architecture with Register File

- Let's assume there are four n-bit registers in processor named **x0**, **x1**, **x2** and **x3**!
 - Let's assume x0 is hardwired to 0 value!
 - Two instruction bits select source operand 1 (**rs1**)
 - Two instruction bits select source operand 2 (**rs2**)
 - Two instruction bits select destination operand (**rd**) where the result from ALU is stored!
- Our instruction is now 8-bit!
 - Instruction[1:0] → opcode
 - Instruction [3:2] → rs1
 - Instruction [5:4] → rs2
 - Instruction [7:6] → rd

Architecture with Register File

- Example
 - `add x2, x1, x3`
 - opcode = 00
 - rs1 = 01
 - rs2 = 11
 - rd = 10
 - Instruction = {10 11 01 00}
 - Similarly, we can have sub, and, or etc.

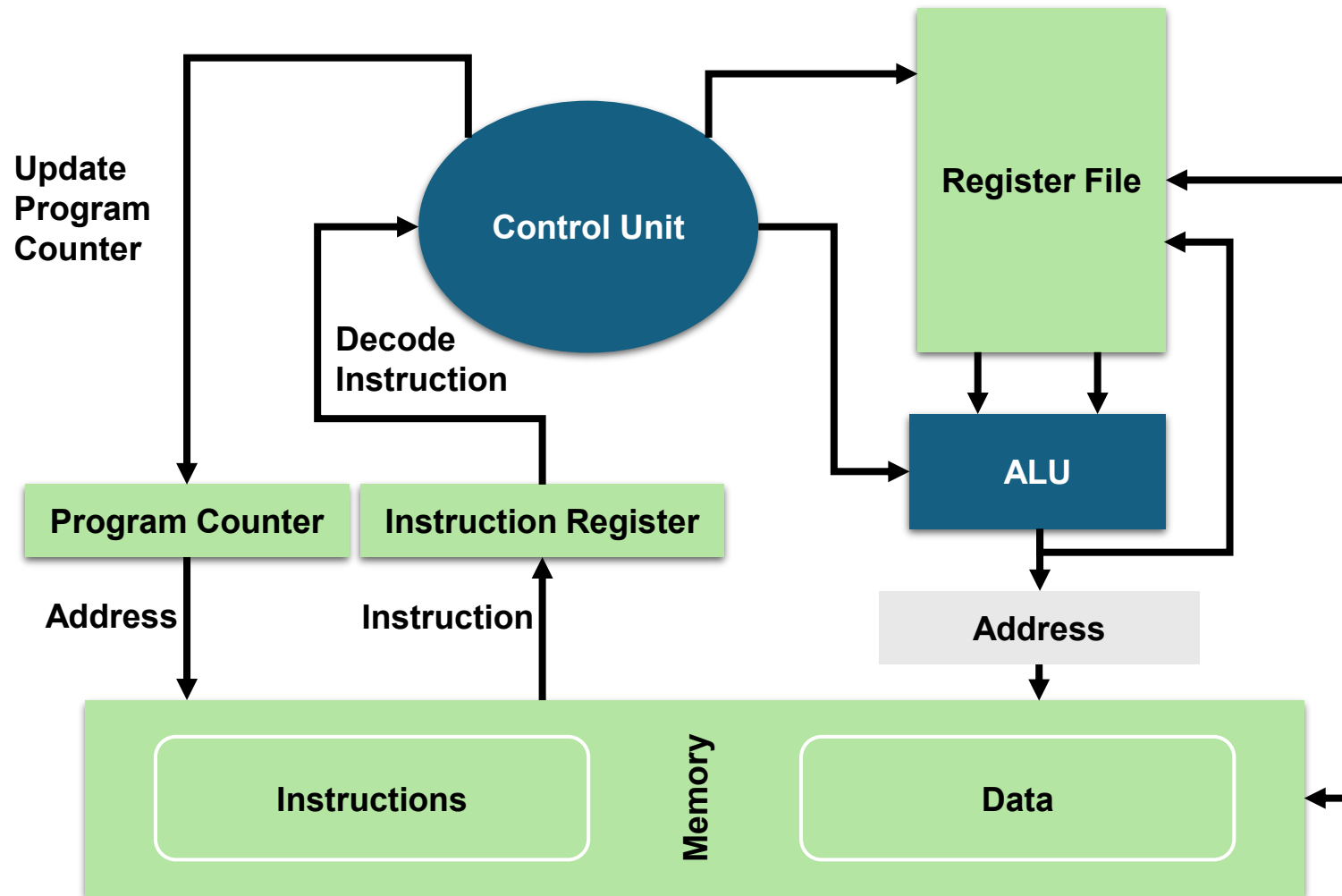
Microarchitecture with Register File



**We will see how more complex processors
are build based on same fundamental
principles!**



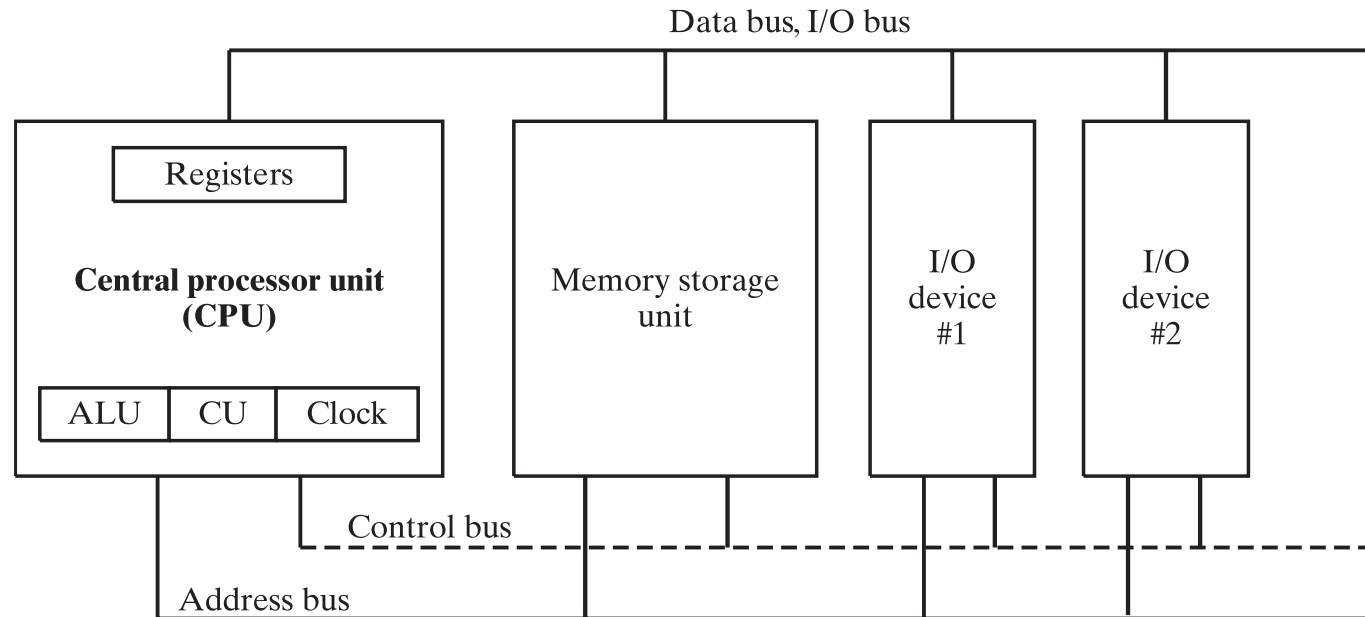
Main Components in a Processor



Interfacing **Microprocessor** with external **Memory and I/O Devices**

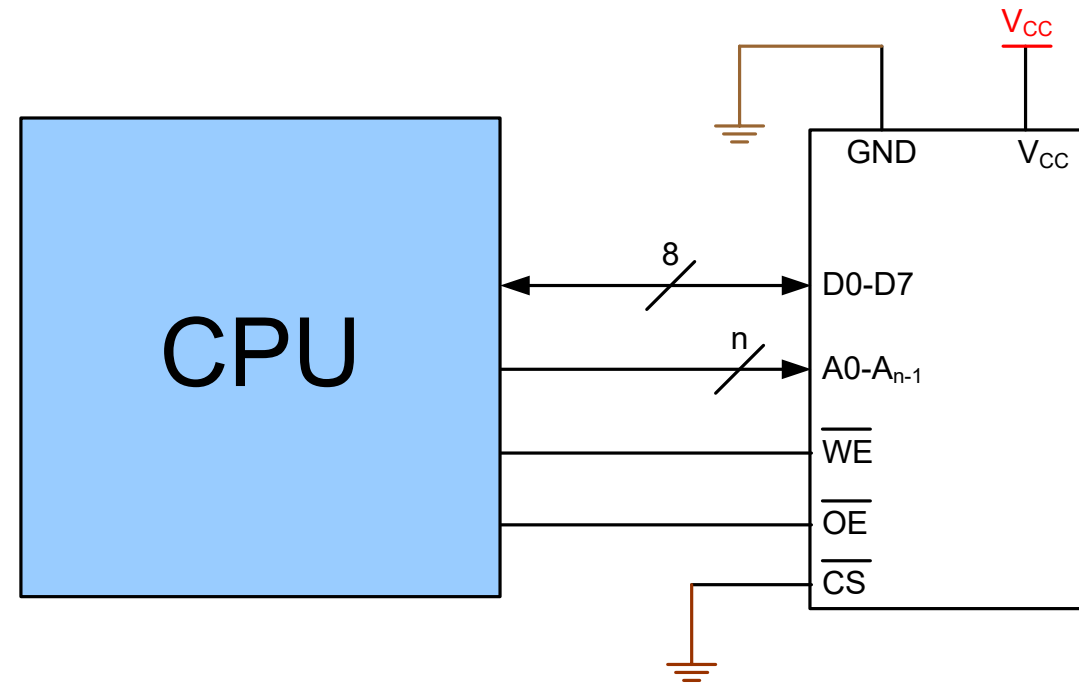


Components of a Computer



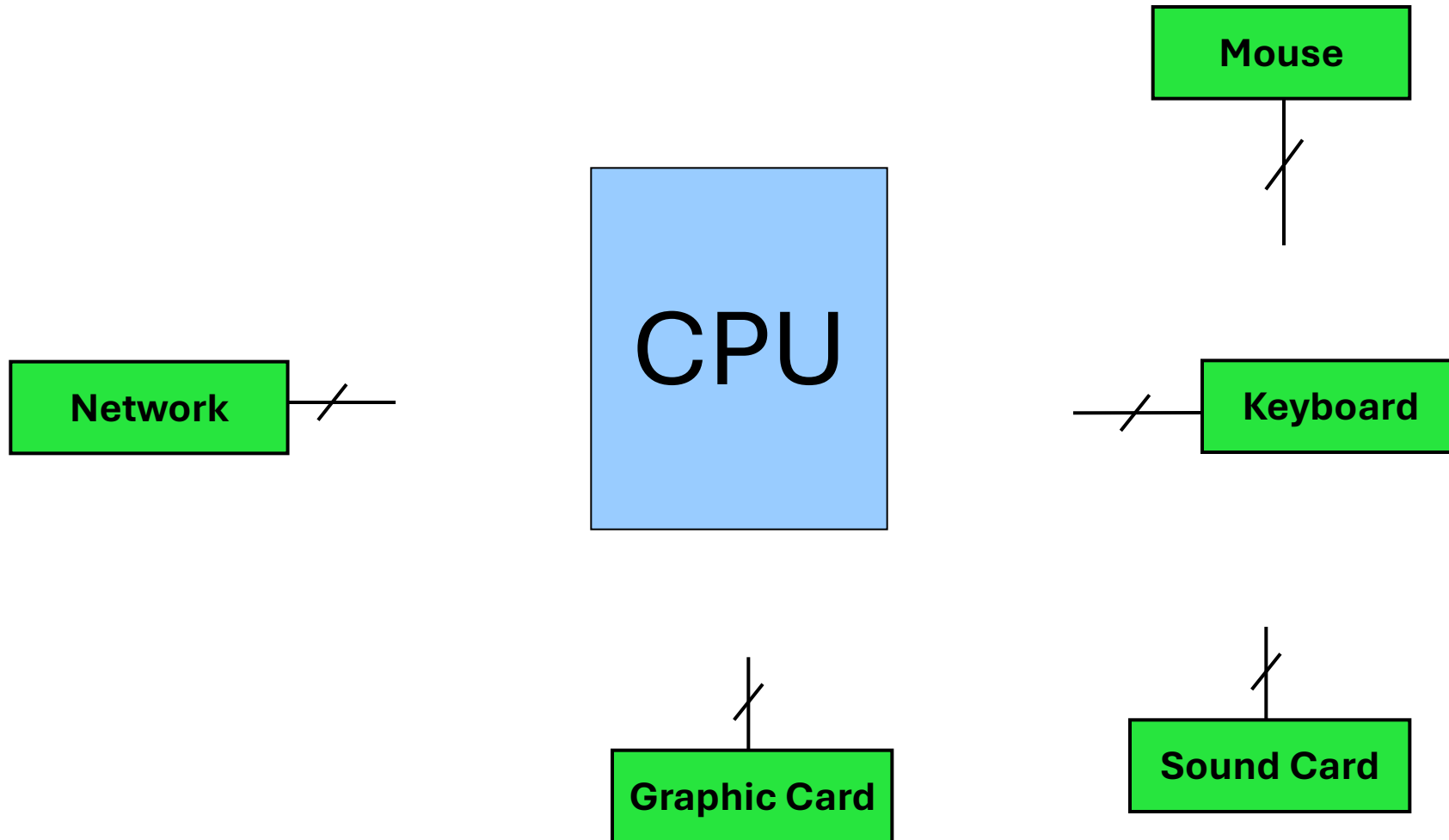
Source: *Assembly Language for x86 Processors*, Kip Irvine

Connecting Off-Chip Memory to CPU

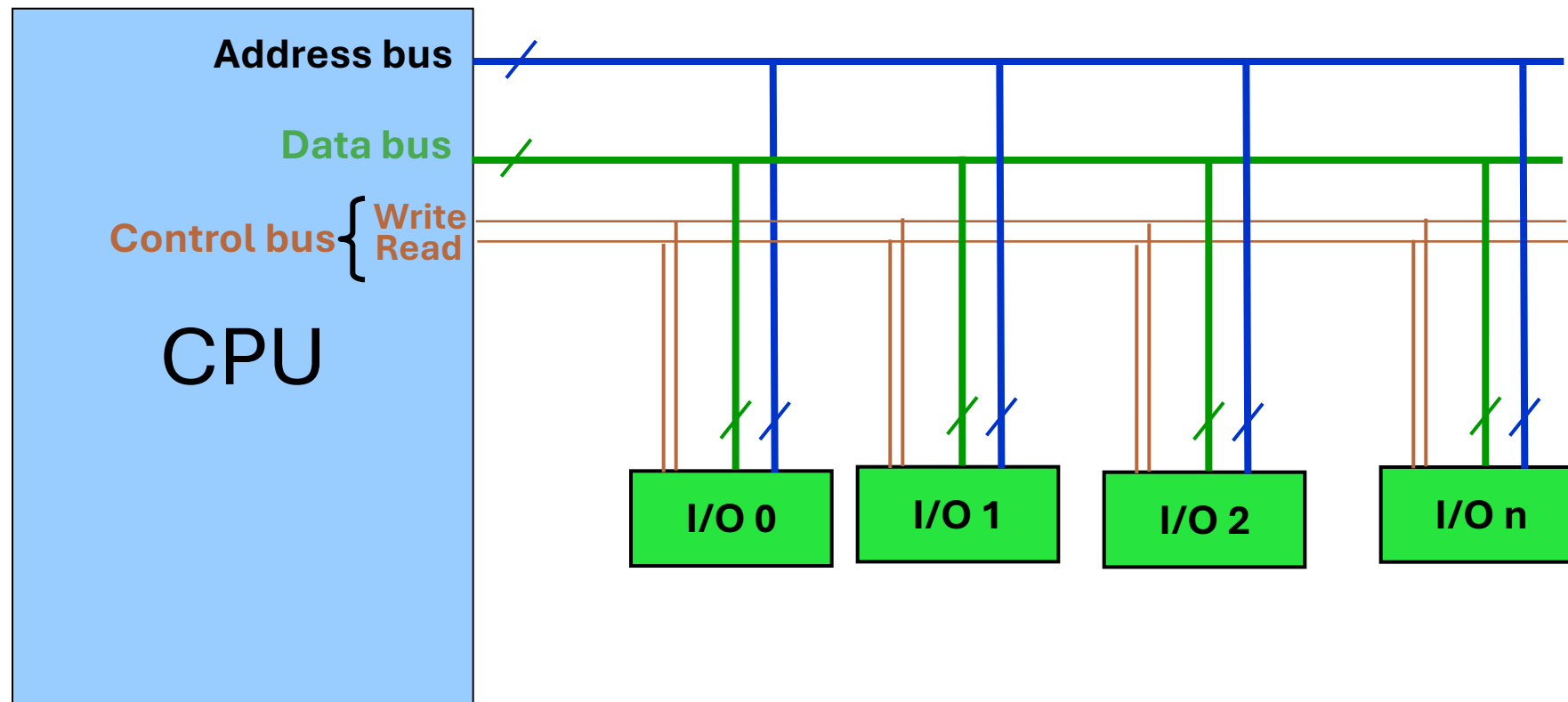


Connecting I/O Devices to CPU

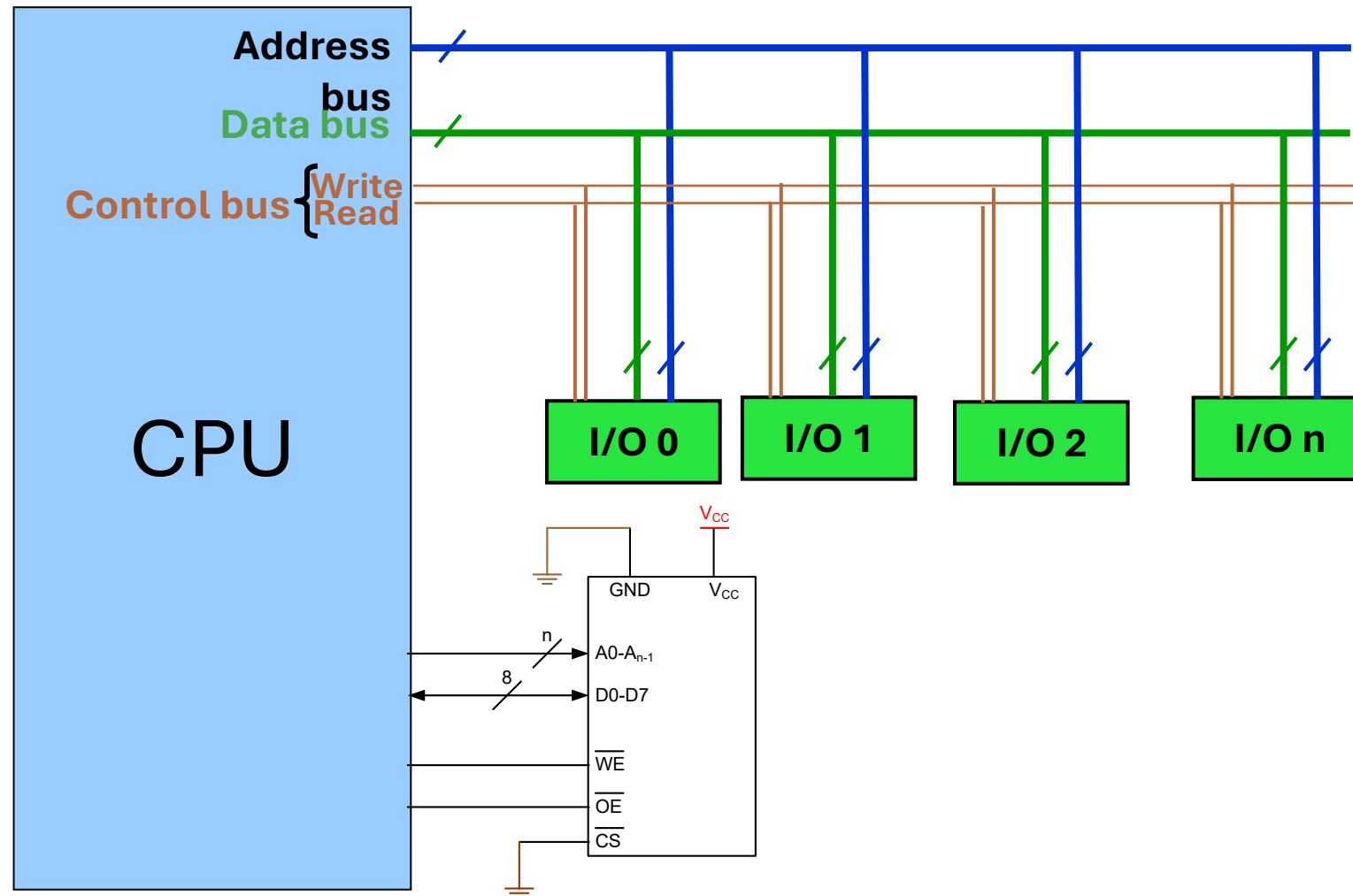
- Lot of Pins?



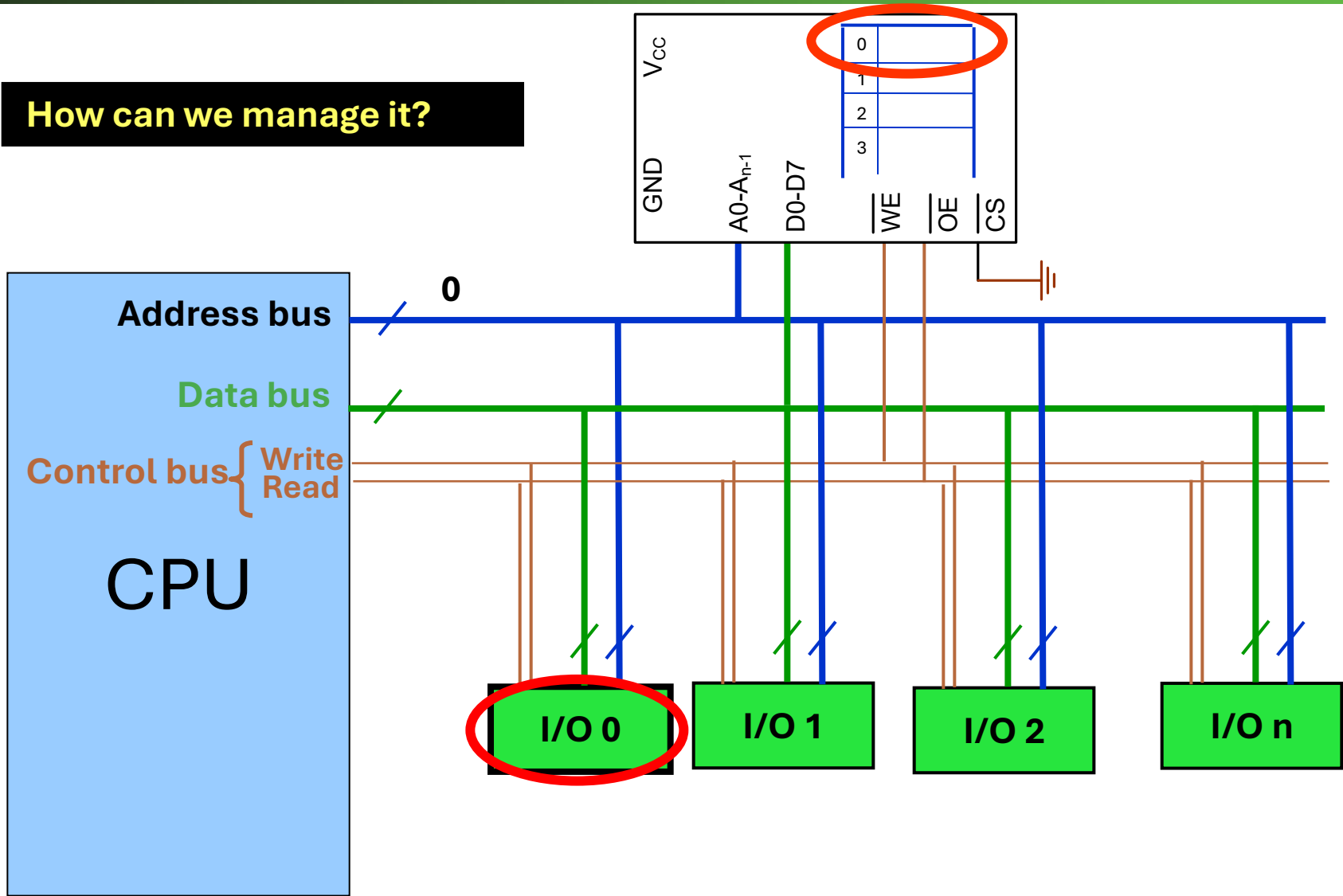
Connecting I/Os to CPU using Bus



Connecting I/Os and Memory to CPU

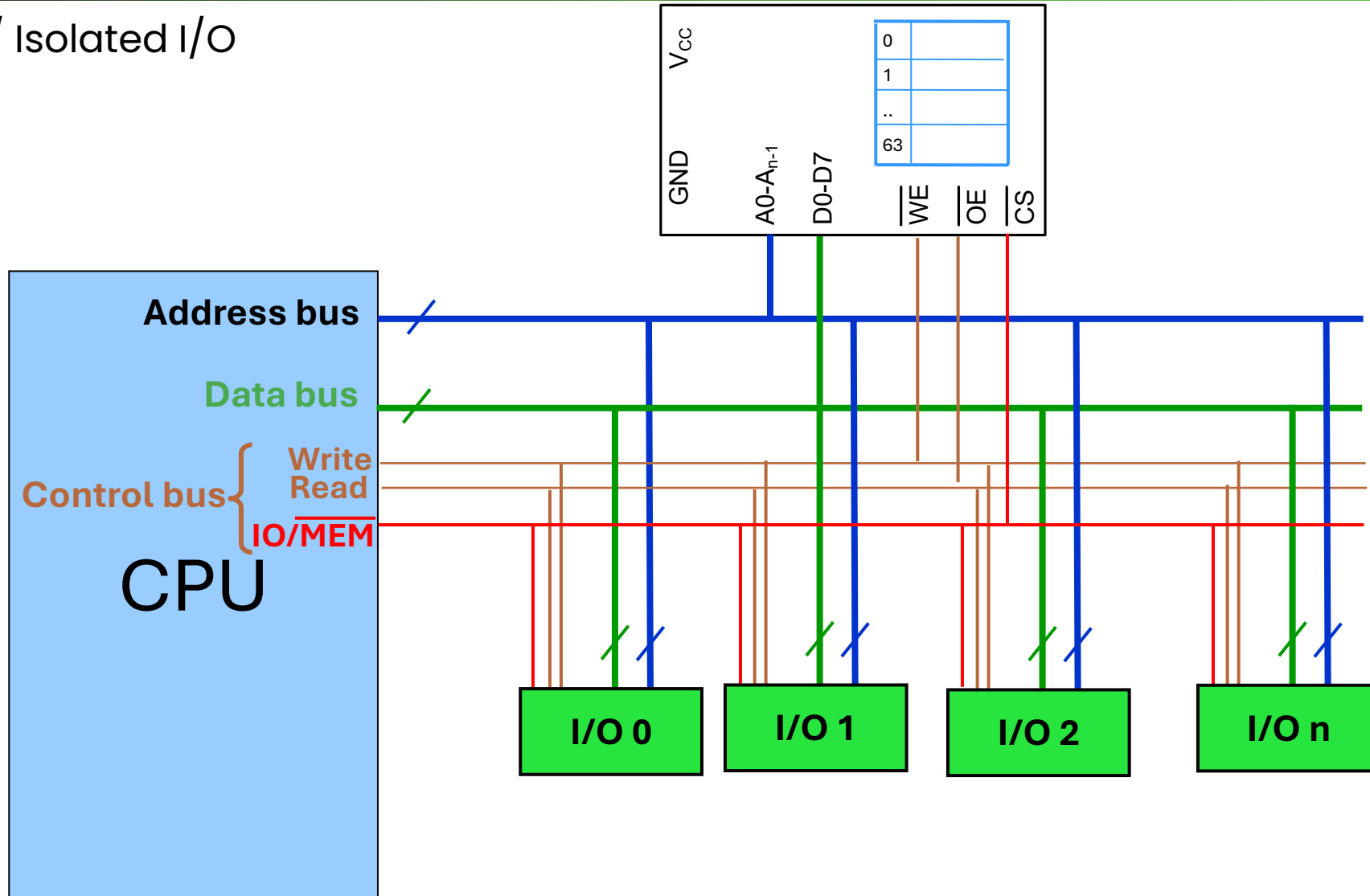


Connecting I/Os and Memory to CPU using Common Bus



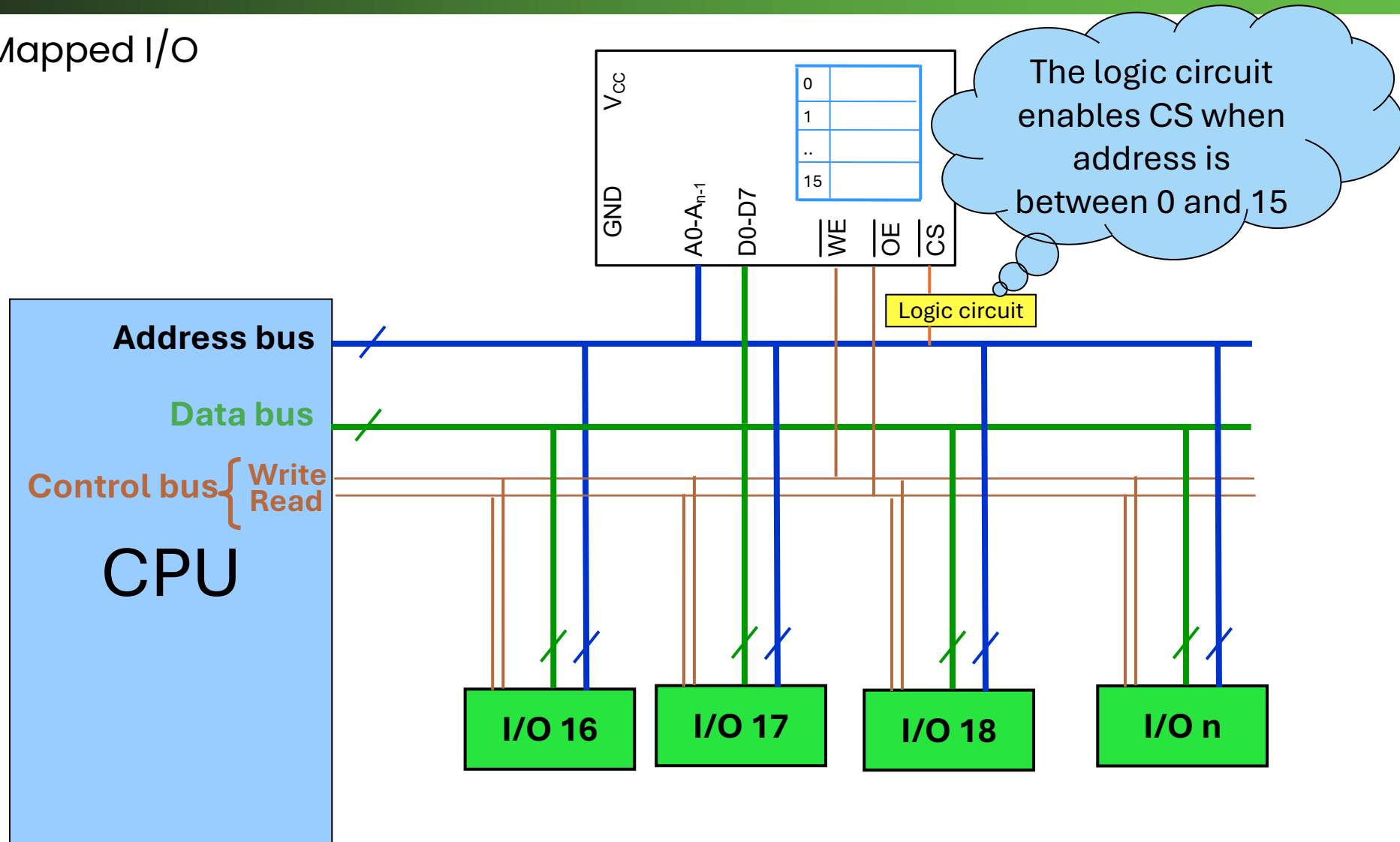
Connecting I/Os and Memory to CPU using Bus

- Peripheral / Isolated I/O



Connecting I/Os and Memory to CPU using Bus

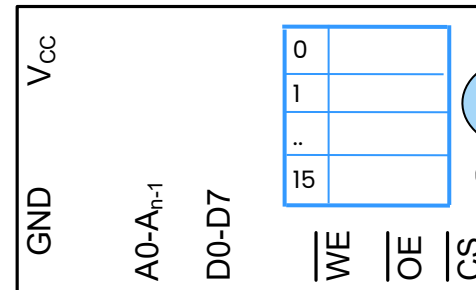
- Memory-Mapped I/O



Connecting I/Os and Memory to CPU using Bus

- Memory-Mapped I/O

How could we make the logic circuit?



The logic circuit enables $\overline{CS}$ when address is between 0 and 15

Logic circuit

Address bus

Solution

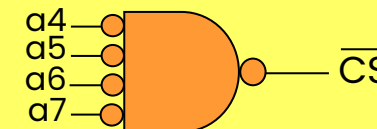
1. Write the address range in binary
2. Separate the fixed part of address
3. Using a NAND, design a logic circuit whose output activates when the fixed address is given to it.

From address 0 →

a7	a6	a5	a4	a3	a2	a1	a0
0	0	0	0	0	0	0	0

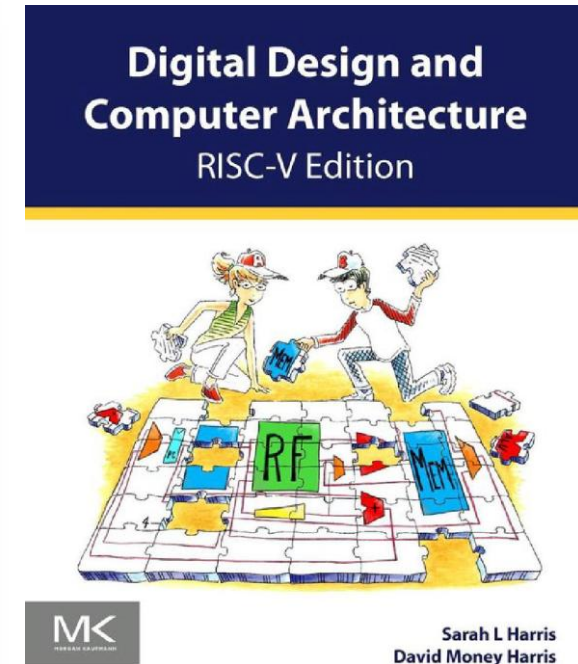
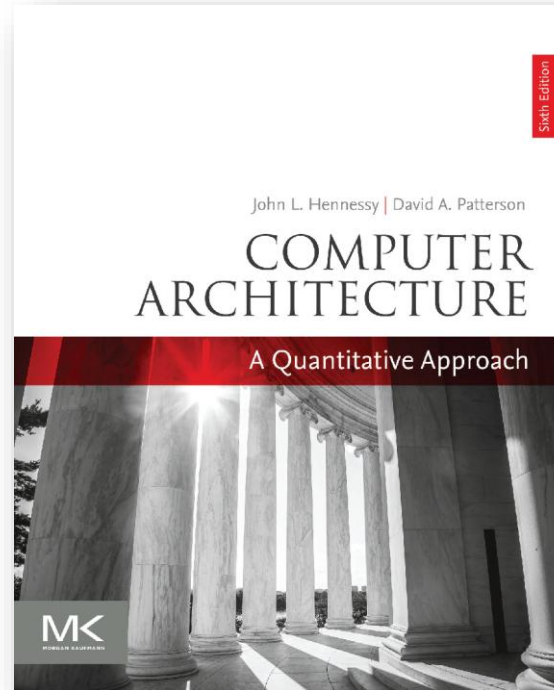
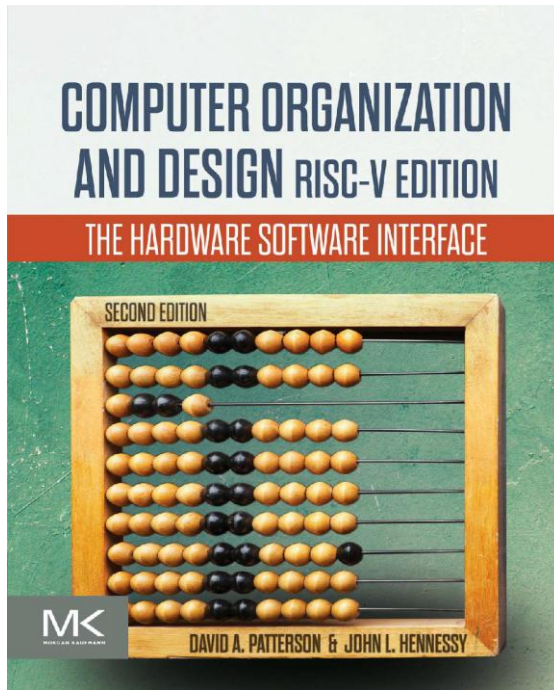
To address 15 →

a7	a6	a5	a4	a3	a2	a1	a0
0	0	0	0	1	1	1	1



Additional Resources to Learn From

- Books



Additional Resources to Learn From

- Courses

- RISC-V
- https://www.youtube.com/watch?v=VEQL5bJeWB0&list=PLbtzT1TYeoMiKup6aoQc3V_d7OvOKc3P5
- Great Ideas in Computer Architecture (Machine Structures), UC Berkley
- <https://cs61c.org/fa24/>
- Digital Design and Computer Architecture, ETH Zurich, Prof. Onur Mutlu
- <https://safari.ethz.ch/digitaltechnik/spring2024/doku.php?id=schedule>
- Computer Architecture, ETH Zurich, Prof. Onur Mutlu
- <https://safari.ethz.ch/architecture/fall2022/doku.php?id=schedule>
- Computer System Architecture, MIT
- <https://csg.csail.mit.edu/6.823S20/index.html>

Additional Resources to Learn From

- Courses
 - SoC 101, Prof. Adam Teman
 - <https://enicslabs.com/academic-courses/soc-101/>
 - Introduction to Computer Architecture, Prof. David Black-Schaffer
 - <https://www.youtube.com/channel/UC0j4jTCkhMLmGwriVbbBtSw/videos>
 - https://www.youtube.com/channel/UCzf_XjloKSf4Ve2fH7xn-3A/playlists
 - Introduction to RISC-V, Linux Foundation
 - <https://training.linuxfoundation.org/training/introduction-to-riscv-lfd110/>
 - RVFPGA Program
 - Learning RISC-V architecture using FPGA!
 - <https://university.imgtec.com/rvfpga/>

Thank You

